

ZKBoost: Zero-Knowledge Verifiable Training for XGBoost

Nikolas Melissaris¹, Jiayi Xu², Antigoni Polychroniadou³, Akira Takahashi³, and
Chenkai Weng²

¹CNRS, IRIF, Université Paris Cité

²Arizona State University

³JPMorgan AI Research & AlgoCRYPT CoE

Abstract

Gradient boosted decision trees, particularly XGBoost, are among the most effective methods for tabular data. As deployment in sensitive settings increases, cryptographic guarantees of model integrity become essential. We present ZKBoost, the first zero-knowledge proof of training (zkPoT) protocol for XGBoost, enabling model owners to prove correct training on a committed dataset without revealing data or parameters. We make three key contributions: (1) a fixed-point XGBoost implementation compatible with arithmetic circuits, enabling instantiation of efficient zkPoT, (2) a generic template of zkPoT for XGBoost, which can be instantiated with any general-purpose ZKP backend, and (3) vector oblivious linear evaluation (VOLE)-based instantiation resolving challenges in proving nonlinear fixed-point operations. Our fixed-point implementation matches standard XGBoost accuracy within 1% while enabling practical zkPoT on real-world datasets.

¹`nikolas@irif.fr`

²`{jiayixu7, chenkai.weng}@asu.edu`

³`{antigoni.polychroniadou, akira.takahashi}@jpmorgan.com`

Contents

1	Introduction	3
1.1	Our contribution	3
1.2	Prior work	4
2	Preliminary and Problem Statement	4
2.1	XGBoost	4
2.2	Zero-Knowledge Proof of Training	5
3	Design of zkPoT for XGBoost	6
3.1	Fixed-Point XGBoost	6
3.2	zkPoT Template for XGBoost Training	7
3.3	ZKP Gadgets for Nonlinear Relations	8
4	Experiments	9
4.1	Fixed-point XGBoost	9
4.2	Zero Knowledge XGBoost-FP	10
5	Conclusion	11
A	Additional Preliminaries	15
A.1	Further Details of XGBoost	15
A.2	Details of Commit-and-Prove Zero Knowledge Proofs	16
A.3	Zero-Knowledge Proof based on Vector Oblivious Linear Evaluation	17
B	Additional Related Work	17
C	Details of Fixed-Point XGBoost Training	18
C.1	Numerical Safeguards in Fixed-Point XGBoost	18
D	Details of Certification Algorithm for XGBoost Training	20
E	Additional Details on ZK XGBoost-FP	23
E.1	Proof of Comparison	23
E.2	Proof of Division and Truncation	25
E.3	Additional Components	25
E.4	Security Analysis	26
F	Details of Certificate for Random Forest Training	27
G	Additional Experiments	28
G.1	Additional Fixed-Point XGBoost Experiments	28
G.2	Discussion for Fixed-Point XGBoost Experimental Results	28
G.3	Comparing with Sparrow	29

1 Introduction

As reliance on ML models grows, concerns about integrity and accountability are increasing. Today, when a model $\mathcal{M}$ is deployed, there is no evidence that it was truly obtained by training on dataset $\mathcal{D}$ with prescribed hyperparameters. This gap is important. A dishonest provider could ship a hand-crafted model, mix in unauthorized data, or skip parts of training. Clients have no way to distinguish such shortcuts from genuine training, because model providers in practice are not willing to reveal proprietary training data or the model parameters due to privacy and intellectual property concerns.

An emerging approach to address this gap is zero knowledge proofs (ZKP) [GMR89], which allows a *prover* to convince a *verifier* of the truth of a statement without revealing anything beyond its validity. In the context of machine learning, it enables *zero knowledge Proof of Training* (zkPoT), allowing a provider (prover) to convince clients (verifier) that “private model $\mathcal{M}$ is the result of a public training algorithm `Train` on cryptographically committed dataset $\mathcal{D}$ ” without exposing either $\mathcal{D}$ or intermediate training states.

There has been tremendous progress recently in zkPoT, especially for neural networks [APKP24, LF25, WSS⁺24], logistic regression [GGJ⁺23, TGM⁺25] and ordinary decision trees [PP24], enabling a number of important applications: (i) *trustworthy ML-as-a-service*, where clients obtain assurance of training integrity; (ii) *decentralized ML*, where smart contracts can reward provable training on public data; and (iii) *compliance with data restrictions*, where clients verify that models were trained on approved data sources (e.g., census data) or that certain protected content (e.g., copyrighted material) was excluded [WSS⁺24]. The last application is particularly powerful: by combining zkPoT with additional ZKP for data-compliance constraints, it prevents providers from arbitrarily manipulating data.

In this work, we turn our attention to zkPoT for gradient boosted decision trees, and in particular the XGBoost library [CG16]. XGBoost has become one of the most widely used methods for structured data, routinely outperforming deep networks on medium-sized tabular datasets [GOV22, SGL24], dominating machine learning competitions, and seeing widespread adoption in finance and healthcare. As such, ensuring integrity and provenance of XGBoost models has wide-reaching implications in practice.

1.1 Our contribution

We propose ZKBoost (illustrated in Figure 1), the first zkPoT protocol allowing a model provider to prove that a classifier was correctly obtained by executing the XGBoost training algorithm on a private dataset. Our technical contributions are threefold:

ZKP-friendly XGBoost implementation. Existing ZKP systems typically support statements expressed as *arithmetic circuits* over finite rings or fields, which does not directly support floating-point arithmetic. Unfortunately, naively translating XGBoost into arithmetic circuits is challenging as it relies on a number of floating-point operations, such as the computation of gradients, Hessians, information gain during split search, leaf weights, (inverse) sigmoid, etc. We thus design a *fixed-point* version of XGBoost, which is particularly well suited for ZKP due to its deterministic semantics, bounded precision, and crucially, its compatibility with arithmetic circuits. We empirically show that our fixed-point implementation achieves nearly identical accuracy to standard floating-point XGBoost across a range of datasets and hyperparameters.

Generic template for zkPoT of XGBoost. We develop a *certification algorithm*, **CertXGB**, which provides an abstraction of arithmetic circuits for efficiently checking whether a given model $\mathcal{M}$ (consisting of the base score and tree ensemble) was obtained by correctly executing our fixed-point XGBoost on an input dataset $\mathcal{D}$. Our **CertXGB** is general: it can be plugged into any general-purpose ZKP backend for arithmetic circuits to instantiate zkPoT of XGBoost. Instead of naively re-executing the XGBoost training procedure, which is inherently sequential, we carefully re-arrange the computation to enable parallel validation. Specifically, while XGBoost training *sequentially* grows trees $T_1, \dots, T_m$ (where each tree depends on predictions from prior trees), our **CertXGB** can validate each tree T_k independently in parallel, and verify the inter-tree dependencies in a separate thread. Moreover, **CertXGB** supports validation of pruning, which is crucial for practical XGBoost to prevent overfitting but has not been addressed in prior zkPoT work on ordinary decision trees [PP24].

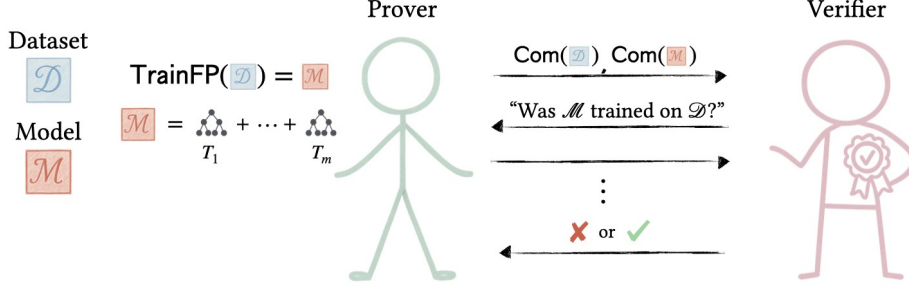


Figure 1: Overview of the ZKBoost protocol for certifying XGBoost training in zero-knowledge. To simplify, we show the owner of the model also owning the dataset but the dataset can belong to some other party in practice.

Instantiation of zkPoT with VOLE-ZKP. We instantiate, implement and evaluate zkPoT of XGBoost on standard datasets, demonstrating practical performance. Our instantiation is built on recent vector oblivious linear evaluation (VOLE)-based ZKP protocols [YSWW21a, WMK16, YH24], which offer fast proving time while maintaining practical communication and verification costs. As a contribution of independent interest, we propose improved ZKP subcomponents for securely proving several non-linear operations on fixed-point numbers, including comparison, division, and truncation. Our instantiation further offers enhanced security by preventing cheating provers from exploiting arithmetic overflows to break soundness of zkPoT.

1.2 Prior work

We defer a detailed discussion of related work to Appendix B. Sparrow [PP24], a zkPoT framework for ordinary decision trees using Gini impurity, lays important groundwork for our work. In particular, their histogram-based certification inspired our design of CertXGB, which initializes node histograms from leaf to root. However, gradient boosting requires more complex inter-tree dependencies and split search than simple decision trees, which hinder naive application of Sparrow. In addition, we address several technicalities that did not arise in Sparrow, including ZKP subprotocols for (1) pruning and binning, (2) range checks of training traces, making sure the prover did not cause arithmetic overflow to break soundness, (3) various fixed-point operations, such as division, gradients, Hessians, sigmoid, etc., (4) handling corner cases such as division by 0. Moreover, we use VOLE-ZK as a backend for faster proving time, while Sparrow uses succinct non-interactive arguments of knowledge (SNARKs) to achieve sublinear verification time.

[TGM⁺25] recently demonstrated a *rejection sampling attack* on zkPoT protocols, allowing a malicious prover to choose the training randomness to bias the model without detection. As our underlying XGBoost training is deterministic, this attack does not apply to our setting.

2 Preliminary and Problem Statement

2.1 XGBoost

We provide an overview of XGBoost, focusing on the algorithmic structure and training workflow. More comprehensive background can be found in Section A.1 and in the original paper [CG16].

Data Structures and Notation We use the bracket notation $[n]$ to denote the set $\{1, 2, \dots, n\}$ for a positive integer n . Datasets are collections $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ with feature vectors $\mathbf{x}_i \in \mathbb{R}^d$ and binary labels $y_i \in \mathcal{Y} = \{0, 1\}$. In this paper, we use the following notation: n as the number of data points in $\mathcal{D}$; m as the number of trees (i.e. weak learners) in $\mathcal{T} = \{T_k\}_{k \in [m]}$; d as the number of features; B as the number of bins; h as the height of T_k ; $N = 2^h$ as the number of leaves in T_k , assuming T_k is a full binary tree (containing dummy nodes as explained below) with height h ; ¹ $N - 1$ as the number of non-leaf nodes in T_k ; γ, λ as the regularization parameters; η as the learning rate.

¹We focus on equal-width pre-binning and a canonical tree structure, which simplifies certification but differs from some production XGBoost features (e.g., quantile histograms or specialized missing-value handling).

To index various quantities, we use k to index trees; i to index data points; f to index features; b to index bins; $\ell \in [2N - 1]$ to index nodes in a tree; $l \in [N]$ to index leaf nodes in a tree; j to index others (e.g., level of a node).

With this notation, each tree T_k is represented as a tuple $(\mathbf{f}_k, \mathbf{t}_k, \mathbf{w}_k) \in [d]^{N-1} \times \mathbb{R}^{N-1} \times \mathbb{R}^N$. For each non-leaf node $\ell \in [N - 1]$, $f_{k,\ell}$ indicates the feature index used for splitting, and $t_{k,\ell}$ indicates the threshold. Each leaf node $l \in [N]$ contains a weight $w_{k,l}$. For each data point $\mathbf{x}_i$, T_k classifies it into a leaf node $l_{k,i}$ by traversing the tree from the root to a leaf node according to the feature values of $\mathbf{x}_i$. XGBoost employs *pruning* to prevent overfitting by marking an internal node as a leaf when the maximum gain is ≤ 0 . However, non-full binary trees leak tree topology, which may be sensitive. To address this, we assume the training algorithm instead sets dummy values $(f_{\text{dum}}, t_{\text{dum}})$ when it would prune: $f_{\text{dum}} \in [d]$ can be an arbitrary feature, and t_{dum} is set to a vacuous constant (e.g., DBL_MIN) so that any sample goes right. See Section C.

Single Tree vs. Boosted Trees. A decision tree maps an input feature vector $\mathbf{x} \in \mathbb{R}^d$ to a prediction by recursively partitioning the feature space using threshold-based splits and assigning a value at each leaf. While such models are simple and interpretable, a single tree often lacks sufficient expressive power.

Gradient boosting addresses this limitation by constructing an *ensemble* of trees $\mathcal{T} = \{T_k\}_{k \in [m]}$ sequentially, where each new tree is trained to correct the residual errors of the current model. XGBoost [CG16] is a widely used and optimized implementation of this paradigm, relying on first and second order loss derivatives to guide split selection and leaf-weight assignment across multiple boosting rounds. As a result, each stage of training depends on the predictions produced by all previous trees, making the training process inherently stateful. This dependency highlights why certifying XGBoost training is substantially more challenging than certifying inference or the construction of a single decision tree.

Numerical safeguards. Practical XGBoost implementations incorporate a number of numerical safeguards. Our fixed-point formulation in Section 3.1 explicitly incorporates such operations into the definition of the training relation itself. A detailed description of these safeguards is provided in Appendix C.1.

2.2 Zero-Knowledge Proof of Training

Zero-Knowledge Proofs. ZKP (of knowledge) allows one party (a *prover*) to prove knowledge of a *secret witness* $\mathbf{w}$ for a *public statement* $\mathbf{x}$ to another party (*verifier*). Such proofs are constructed for a concrete NP relation $\mathcal{R}$, describing the relationship between $\mathbf{x}$ and $\mathbf{w}$. Formally, ZKP for an NP relation $\mathcal{R}$ is a tuple of interactive Turing machines $(\mathcal{P}, \mathcal{V})$, where $\mathcal{P}$ is prover and $\mathcal{V}$ is verifier. Then $\mathcal{P}$ and $\mathcal{V}$ interact with each other, where both $\mathcal{P}$ and $\mathcal{V}$ take $\mathbf{x}$ as common inputs, and $\mathcal{P}$ additionally takes $\mathbf{w}$ as a private input. At the end of interaction, $\mathcal{V}$ outputs a binary b .

Proof systems that are used in verifiable ML typically require the following security properties: For an NP relation $\mathcal{R}$, they must provide *completeness* (i.e., if prover and verifier follow the protocol with input $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$, verifier always accepts), *knowledge soundness* (i.e., if verifier accepts, then it must be that prover owns a valid witness $\mathbf{w}$ satisfying given NP relation w.r.t. statement $\mathbf{x}$), and *zero knowledge* (i.e., the transcript of the interaction between the prover and the (malicious) verifier leaks nothing except that there exists a witness $\mathbf{w}$ such that $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$).

Commitments. Cryptographic commitments are often an important building block of ZKPs, ensuring that the prover is bound to a specific value while keeping it hidden from the verifier. Formally, a commitment scheme is defined as an algorithm Com that allows committing to a message m with randomness r : $c \leftarrow \text{Com}(m; r)$. It must satisfy two properties: *Binding*: It is infeasible to open c to two different messages, i.e., it is computationally hard to find (c, m, r, m', r') such that $c = \text{Com}(m; r) = \text{Com}(m'; r') \wedge m \neq m'$, and *Hiding*: c reveals nothing about m . As these are standard building blocks in cryptography, we refer the reader to [Gol01] for more details.

Zero-Knowledge Proof of Training (zkPoT). Once ZKP and commitment schemes with the above properties are given, we can define a secure zkPoT for a training algorithm TrainXGB that takes a dataset $\mathcal{D}$ as input and outputs a model $\mathcal{M} = \text{TrainXGB}(\mathcal{D})$. For training verification, we define the relation

$$R_{\text{xgb}} = \left\{ ((\mathcal{C}_{\mathcal{M}}, \mathcal{C}_{\mathcal{D}}), (\mathcal{M}, \mathcal{D}, r, \rho)) : \begin{array}{l} \mathcal{C}_{\mathcal{M}} = \text{Com}(\mathcal{M}; r) \\ \mathcal{C}_{\mathcal{D}} = \text{Com}(\mathcal{D}; \rho) \\ \mathcal{M} = \text{TrainXGB}(\mathcal{D}) \end{array} \right\} \quad (1)$$

A zkPoT protocol ensures that: (1) Completeness: An honest prover with dataset $\mathcal{D}$ and model $\mathcal{M} = \text{TrainXGB}(\mathcal{D})$ can produce a valid proof. (2) (Knowledge) Soundness: Any prover that outputs a valid proof must “know” such a dataset $\mathcal{D}$ and a valid model $\mathcal{M}$ derived from $\mathcal{D}$ via **TrainXGB**. (3) Zero-Knowledge: The proof leaks no information about $\mathcal{D}$ or $\mathcal{M}$ beyond the commitments. To formalize the above properties, we use the so-called *simulation-based security* defined in terms of a commit-and-prove ideal functionality (see Appendix A.2 for details).

3 Design of zkPoT for XGBoost

3.1 Fixed-Point XGBoost

Zero-knowledge proof systems operate over finite rings or fields, making floating-point arithmetic impractical as it involves rounding, exponents, and platform-dependent nondeterminism that are difficult to represent succinctly. We therefore redesign XGBoost so that all quantities are represented in *fixed-point arithmetic* with an implicit global scaling factor **scale**. This section introduces the representation, the proof-friendly approximations to the sigmoid and log-odds functions, and the implications for training correctness. Further details and the complete pseudocode **TrainXGB** are provided in Appendix C.

Fixed-Point Representation. Each real value $x \in \mathbb{R}$ is represented by an integer

$$\tilde{x} = \lfloor x \cdot \text{scale} \rfloor,$$

and all arithmetic is carried out over integers. Addition and subtraction operate directly on these integers, while multiplication and division are defined as:

$$\widetilde{x \cdot y} = \left\lfloor \frac{\tilde{x} \cdot \tilde{y}}{\text{scale}} \right\rfloor, \quad \frac{\tilde{x}}{\tilde{y}} = \left\lfloor \frac{\tilde{x} \cdot \text{scale}}{\tilde{y}} \right\rfloor. \quad (2)$$

The division by **scale** (or multiplication by it) is performed *over integers*, and the result is floored to remain within the integer domain. This keeps every operation deterministic, ensuring that the training relation forms a well-defined NP relation suitable for zero-knowledge proofs. In the following, we omit the tilde notation and treat all variables as fixed-point integers with implicit scaling.

Proof-Friendly Sigmoid and Log-Odds. In XGBoost, the score z in the log-odds space for each boosting round gets converted into probability via sigmoid $\sigma(z) = 1/(1 + e^{-z})$, and the base score is conversely initialized by taking the log-odds $\sigma^{-1}(p) = \log(p/(1 - p))$. These are not proof-friendly due to the exponential and logarithmic operations, requiring floating-point arithmetic. We replace them with fixed-point variants that approximate these functions accurately in the range $[-2, 2]$:

$$\text{SigmoidWideFP}(z) = \begin{cases} 0, & z \leq -2, \\ \frac{z+2}{4}, & -2 < z < 2, \\ 1, & z \geq 2, \end{cases} \quad (3)$$

$$\text{LogitFromProbFP}(p) = 2 \left(u + \frac{u^3}{3} + \frac{u^5}{5} \right), \quad u := 2p - 1. \quad (4)$$

where all powers and divisions are evaluated according to Eq. (2). Equation (3) is linear within $[-2, 2]$ and saturates outside this range, avoiding costly exponentials. Equation (4) is a truncated Taylor expansion of the $\tanh^{-1}$ function ($\text{atanh}(u) = u + u^3/3 + u^5/5 + \dots$), providing a smooth and invertible log-odds mapping compatible with integer scaling.

Design Implications. These substitutions render the entire training process discrete and reproducible. All gradient $g_i = p_i - y_i$, Hessian $h_i = p_i(1 - p_i)$ for each data point $(\mathbf{x}_i, y_i)$, their (partial) sums G, G_L, G_R, H, H_L, H_R , and gain computations—previously involving floating-point divisions—are now expressed as scaled integer ratios:

$$\text{gain} = \frac{1}{2} \left(\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right) - \gamma, \quad (5)$$

where each division follows the fixed-point rule in Eq. (2). Additional numerical safeguards such as feature binning, probability clipping, and bounded leaf weights are applied as in standard XGBoost but are now part of the certified relation itself rather than ad-hoc implementation details.

Empirically (see Section 4), this fixed-point formulation reproduces floating-point XGBoost accuracy within statistical variation (less than 1%), while enabling proof-friendly training semantics suitable for succinct zero-knowledge proofs.

3.2 zkPoT Template for XGBoost Training

We first introduce our certification algorithm **CertXGB** for verifying the correctness of XGBoost training, and use it to describe a generic zkPoT template. Essentially, **CertXGB** represents a circuit that takes a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ and a claimed model $\mathcal{M} = (\mathcal{T}, z_0)$ as input, and then outputs 1 (“accept”) or 0 (“reject”) by checking that the model $\mathcal{M}$ is correctly produced by training on $\mathcal{D}$ using our fixed-point XGBoost training **TrainXGB** (Section 3.1 and C). The complete procedure is deferred to Appendix D.

Overview Recall that the original XGBoost (Appendix A.1) essentially proceeds as follows:

1. Initialize the base score (logit) z_0 from $\mathcal{D}$.
2. **Iterative Tree Building.** For each boosting round $k = 1, \dots, m$, it grows a new tree T_k from root to leaves by (a) computing gradients and Hessians, (b) finding the best splits maximizing the gain, and (c) computing leaf weights. Finally, it computes the updated scores $\mathbf{z}_k = (z_{k,i})_{i=1}^n$.

Instead of naively re-executing the above training procedure, **CertXGB** takes advantage of the fact that the final model $\mathcal{M} = (\mathcal{T}, z_0)$ is provided as input. At a high level, **CertXGB** verifies that each tree T_k in $\mathcal{T}$ is correctly built by shuffling the order of operations in Step 2 above: it first evaluates T_k on every data point $\mathbf{x}_i$ to determine the reached leaf indices $l_{k,i}$ and update the score $z_{k,i}$, and then reconstructs the histograms of gradients and Hessians at each node using these indices and scores from leaves to root. Finally, it checks that each split in T_k maximizes the gain computed from the reconstructed histograms, and that each leaf weight is consistent with the corresponding histogram.

In slightly more detail, **CertXGB** verifies that $(\mathcal{T}, z_0)$ is the output of **TrainXGB** on input $(\mathbf{x}, \mathbf{y})$, following the steps below. Note that Step 3 can be parallelized, because each tree validation is independent of others and inter-tree dependencies can be handled separately in Step 2. The reader may jump to the detailed procedure by clicking each subroutine.

1. **ValidateLogit:** To validate the base logit z_0 , **CertXGB** computes the initial prediction probability $p = \frac{1}{n} \sum_{i=1}^n y_i$, centers it to $u = 2p - 1$, and then computes $z'_0 = 2 \cdot \left(u + \frac{u^3}{3} + \frac{u^5}{5}\right)$ using a truncated Taylor series of atanh with fixed-point operations. The circuit asserts that $z_0 = z'_0$ and sets $z_{0,i} = z_0$ for all $i \in [n]$.
2. **Initialize intermediate scores.** For each tree $k \in [m]$ and each data point $i \in [n]$, **CertXGB** evaluates the tree $T_k = (\mathbf{f}_k, \mathbf{t}_k, \mathbf{w}_k)$ on $\mathbf{x}_i$ to compute the reached leaf $l_{k,i} \in [N]$, and updates the score $z_{k,i} \leftarrow z_{k-1,i} - \eta \cdot w_{k,l_{k,i}}$.
3. **Tree Validation.** For each tree $k \in [m]$, do:
 - (a) **ValidateInference:** Validate that the computed leaf indices $\mathbf{l}_k = (l_{k,i})_{i=1}^n$ are correct by checking that each $l_{k,i}$ is consistent with the feature splits $(\mathbf{f}_k, \mathbf{t}_k)$ and the input $\mathbf{x}_i$.
 - (b) **InitHists:** Computes histograms (G_k, H_k) by aggregating gradients and Hessians over bins and leaves using the bin indices and leaf indices $\mathbf{l}_k$. Here, when computing the gradients $g_i = p_i - y_i$ and Hessians $h_i = p_i(1 - p_i)$, **CertXGB** uses the prediction probabilities $p_i = \text{SigmoidWideFP}(z_{k-1,i})$ derived from the scores $\mathbf{z}_{k-1}$.
 - (c) **ValidateLeafWeights:** Validate the leaf weights $\mathbf{w}_k$ by checking consistency with the histograms (G_k, H_k) .
 - (d) **ValidateSplits:** Validate the splits $(\mathbf{f}_k, \mathbf{t}_k)$ by ensuring (1) the gain derived from each split is maximal, or (2) if the gain is ≤ 0 (i.e., pruning condition is met), the corresponding node contains dummy values.

In Appendix D, we prove the following claim:

Claim 1 *The certificate algorithm **CertXGB** verifies the correctness of XGBoost training. That is, for any $\mathcal{D} = (\mathbf{x}, \mathbf{y})$, $\mathcal{T} = \{T_k\}_{k \in [m]}$ and z_0 , $\text{CertXGB}(\mathbf{x}, \mathbf{y}, \mathcal{T}, z_0) = 1$ if and only if $(\mathcal{T}, z_0) =$*

TrainXGB($\mathbf{x}, \mathbf{y}$).

With CertXGB in place, we can now describe a generic template of ZKP for relation R_{xgb} (1):

Protocol 1: zkPoT for XGBoost

Parameters: Number of points n , features d , trees m , bins B , leaves $N = 2^h$ at depth h , learning rate η , regularizers λ, γ .

Public Input: Auxiliary commitments $\text{com}_{\mathcal{D}}$ to training data $\mathcal{D} = (\mathbf{x}, \mathbf{y})$, $\text{com}_{\mathcal{T}}$ to trees $\mathcal{T} = \{T_k\}_{k \in [m]}$ with $T_k = (\mathbf{f}_k, \mathbf{t}_k, \mathbf{w}_k)$, and com_z to base logit z_0 .

Private Input of $\mathcal{P}$: Fixed-point training data $(\mathbf{x}, \mathbf{y})$, the fitted tree ensemble $\mathcal{T}$, auxiliary traces, and commitment randomness.

Validate Commitments. $\mathcal{P}$ and $\mathcal{V}$ run a subprotocol for proving knowledge of commitment openings: $\text{com}_{\mathcal{T}} = \text{Com}(\mathcal{T}; r_{\mathcal{T}})$, $\text{com}_z = \text{Com}(z_0; r_z)$, $\text{com}_{\mathcal{D}} = \text{Com}(\mathcal{D}; r_{\mathcal{D}})$.

Validate Binary Input. $\mathcal{P}$ and $\mathcal{V}$ run a subprotocol for asserting $y_i \in \{0, 1\}, \forall i \in [n]$

Compute Edges and binID. $\mathcal{P}$ and $\mathcal{V}$ run a subprotocol for $(\text{edges}, \text{binID}) \leftarrow \text{PreBinFP}(\mathbf{x}, B)$, where edges are bin left edges and binID are bin indices for each feature of each sample; expose authenticated $(\text{edges}, \text{binID})$ to be reused throughout.

Validate Base Logit. $\mathcal{P}$ and $\mathcal{V}$ run a subprotocol for $\text{ValidateLogit}(\mathbf{y}, z_0) = 1$ to validate the initial logit $z_0 = 2 \cdot \text{atanh}(2p - 1)$ with $p = \frac{1}{n} \sum_i y_i$. They then define the replicated vector $\mathbf{z}_0 = (z_{0,i})_{i=1}^n$ with $z_{0,i} = z_0$.

Initialize Intermediate Scores. For each tree $k \in [m]$ and data point $i \in [n]$, $\mathcal{P}$ computes the reached leaf $l_{k,i} \in [N]$ by evaluating T_k on $\mathbf{x}_i$, updates the score $z_{k,i} \leftarrow z_{k-1,i} - \eta \cdot w_{k,l_{k,i}}$, and defines $\mathbf{z}_k \leftarrow (z_{k,i})_{i=1}^n$. Let $\mathbf{l}_k = (l_{k,i})_{i=1}^n$.

For each tree $k \in [m]$, perform the following:

Validate Inference. $\mathcal{P}$ and $\mathcal{V}$ run a subprotocol for $\text{ValidateInference}(\mathbf{x}, \mathbf{f}_k, \mathbf{t}_k, \mathbf{l}_k) = 1$ to validate $\mathbf{l}_k$.

Initialize Node Histograms. $\mathcal{P}$ and $\mathcal{V}$ run a subprotocol for $(G_k, H_k) \leftarrow \text{InitHists}(\mathbf{z}_{k-1}, \mathbf{y}, \text{binID}, \mathbf{l}_k)$, where $G_k[f][\ell][b]$ and $H_k[f][\ell][b]$ are the sum of gradients and Hessians of samples in node ℓ whose feature f falls into bin b . These are computed by summing over samples in each leaf l and propagating them up to the root.

Validate Leaf Weights. $\mathcal{P}$ and $\mathcal{V}$ run a subprotocol for $\text{ValidateLeafWeights}(G_k, H_k, \mathbf{w}_k) = 1$ to validate all leaf weights $\mathbf{w}_k$.

Validate Tree Splits. $\mathcal{P}$ computes $(\mathbf{f}_k, \mathbf{t}_k) \leftarrow (f_{k,\ell}^*, t_{k,\ell}^*)_{\ell \in [N-1]}$, where $(f_{k,\ell}^*, t_{k,\ell}^*)$ for ℓ -th internal node is a split maximizing the gain derived from G_k, H_k . $\mathcal{P}$ and $\mathcal{V}$ run a subprotocol for $\text{ValidateSplits}(G_k, H_k, \mathbf{f}_k, \mathbf{t}_k, \mathbf{f}_k^*, \mathbf{t}_k^*, \text{edges}) = 1$ to check that (1) the gain derived from each split in $(\mathbf{f}_k, \mathbf{t}_k)$ is maximal, or (2) if the gain is ≤ 0 , the corresponding node contains dummy values.

3.3 ZKP Gadgets for Nonlinear Relations

Previous sections build a ZKP-friendly XGBoost by fixed-point computation, polynomial approximation, and algorithm transform. Next, we tackle the proof of low-level nonlinear functions that dominate the overhead of zkPoT for XGBoost-FP. In this part, $\llbracket \cdot \rrbracket$ denotes intermediate values committed by $\mathcal{P}$, we have

- The proof of comparison between two signed fixed-point numbers, denoted as $\llbracket s \rrbracket \leftarrow \mathbf{1}\{\llbracket x \rrbracket < \llbracket y \rrbracket\}$. It proves $s = 1$ if $x < y$, and $s = 0$ otherwise.
- The proof of division between two fixed-point numbers, denoted as $\llbracket z \rrbracket \leftarrow \llbracket x \rrbracket / \llbracket y \rrbracket$.
- The proof of truncation after the multiplication of two fixed-point numbers, denoted as $\llbracket y \rrbracket \leftarrow \text{Trunc}(\llbracket x \rrbracket)$, where $\llbracket y \rrbracket = \lfloor \llbracket x \rrbracket / 2^f \rfloor$ assume f -bit precision.

Existing approaches do not satisfy our goal due to efficiency, vulnerability or lack of support for fixed-point arithmetic. We design rigorous constraint systems to verify these operations with the focus of both efficiency and soundness. We sketch our main contribution in a high level and defer the formalized description and the instantiation of other components to Appendix E.

Comparison. We improve the idea of [HCL⁺24] to prove the comparison relation by bits-decomposition. Note that we use a signed representation. Hence, we have $\mathbf{1}\{\llbracket x \rrbracket < \llbracket y \rrbracket\} = \text{MSB}(\llbracket x - y \rrbracket)$, where MSB

refers to the most significant bit of $z = x - y$. The main task is shifted to proving $\llbracket s \rrbracket = \text{MSB}(\llbracket z \rrbracket)$ given committed $(\llbracket z \rrbracket, \llbracket s \rrbracket)$. We describe prior approach in detail in Appendix E.1, and focus on improving its consistency checks on invalid bits-decomposition.

We abstract out the problem in the following way. The proof of bit-decomposition requires $\mathcal{P}$ to commit to groups of bits $(z_0, \dots, z_{t-1})$ each of d -bit, and show that $z = 2^{n-1} \cdot s + \sum_{i=0}^{t-1} 2^{id} \cdot z_i$. However, a cheating $\mathcal{P}$ may commit to incorrect $(\tilde{z}_0, \dots, \tilde{z}_{t-1})$ such that $z + p = 2^{n-1} \cdot s' + \sum_{i=0}^{t-1} 2^{id} \cdot \tilde{z}_i$, which would completely flip the MSB s .

Our solution employs a Mersenne prime in the form of $p = 2^n - 1$. The only chance a cheating $\mathcal{P}$ has is when $z = 0$, it may claim $s = 1$ and $\tilde{z}_i = 2^d - 1$ for all $i \in [0, t)$. By defining $\llbracket w \rrbracket = \mathbf{1}\{\llbracket z \rrbracket \neq 0\}$, we observe that $\text{MSB}(z) = \begin{cases} 0 & \text{if } z = 0, w = 0 \\ s & \text{if } z \neq 0, w = 1 \end{cases}$. It implies that $(1 - w) \cdot s = 0$ should always be true.

Additionally, we employ the non-equality-zero check from [PHGR13] to prove $\llbracket w \rrbracket = \mathbf{1}\{\llbracket z \rrbracket \neq 0\}$. Our solution to address the soundness issue only requires committing 2 extra values and proving 3 multiplicative relations, while the previous work almost costs $2 \times$ [HCL⁺24].

Division and Truncation. Proof of division takes inputs $(\llbracket x \rrbracket, \llbracket y \rrbracket, \llbracket z \rrbracket)$ and proves that $z = \lfloor x/y \rfloor$. In our XGBoost-FP, the numerators and denominators may range from 0 to nearly $p/2^f$. Previous proofs of division are not suitable since they either only work for bounded small values [LXZ21, PP24] or requires thousands of logical constraints [WYX⁺21a].

To prove the relation, we leverage the existence of a residue $r \in [0, y)$ such that $x = z \cdot y + r$. Additionally, two range checks are required to maintain the soundness: $r \in [0, y)$, and $z \in [0, \lfloor p/y \rfloor]$. The first check ensures r is a residue. The second check is needed to prevent the similar wrap-around issue happened in the proof of MSB. Namely, there could be many possible pairs of (z', r') such that $x = z' \cdot y + r' \bmod p$ is z' . The proof should ensure a z that satisfies $x = z \cdot y + r$ without modulo p . Additionally, our design works for both positive and negative inputs.

The truncation is a simplified division with public and positive denominators $y = 2^f$. Notably, the range check of $r \in [0, 2^f)$ can be implemented by a table lookup. Also, If a Mersenne prime $p = 2^n - 1$ is used, we have $\lfloor (p-1)/2^f \rfloor = 2^{n-f} - 1$ so that $z \in [0, 2^{n-f} - 1]$ is simplified in the similar way.

Global Range Checks for Soundness. To ensure the soundness, global range check is needed for every intermediate values that are committed during the proof. This is because a cheating prover may leverage the overflow to prove a wrong computation. We implement the range checks by the proof of comparison. However, we closely examine the statement, analyze the upper bound of all intermediate values, and apply range checks only when needed. Details are deferred to Appendix E.3.

Instantiation and Security. Our ZKP protocol can be instantiated by any general-purpose proof system. To achieve the maximum prover efficiency, we realize it in the VOLE-ZK framework [YSWW21a]. We state our main theorem of security below and provide details and security analysis in Appendix E.4.

Theorem 1. *Define the relation by the proof of fixed-point XGBoost Training described in Protocol 1, it securely realizes $\mathcal{F}_{\text{CP}}$ in the $(\mathcal{F}_{\text{CVOLE}}, \mathcal{F}_{\text{RAMZK}})$ -hybrid model.*

4 Experiments

We implement both plaintext and ZK XGBoost-FP, and measure their performance. Firstly, we analyze how fixed-point representation affects the accuracy and efficiency of the XGBoost training in plaintext. Secondly, we explore the ZK XGBoost regarding the proving time, bandwidth usage, and memory usage.

4.1 Fixed-point XGBoost

Setup. We compare our fixed-point arithmetic gradient boosting implementation (“Fixed”) against XGBOOST (scikit-learn API), using identical hyperparameters (depth, `bin`=128, $\eta = 0.3$, $\lambda = 1$, $\gamma = 0$) and identical train/test splits per configuration. We evaluate *equal-thread* runs with both OpenMP and BLAS pinned to one thread. Timing measures end-to-end training plus prediction.

Datasets. We use four standard binary classification benchmarks: **Breast Cancer** ($n = 569, d = 30$), **Default of Credit Card Clients** ($n = 30001, d = 23$), two **Covertypes** subsets (converted for

Table 1: Accuracy and runtime of our fixed-point algorithm vs XGBoost. Column header: Parameters (dataset, depth h , # of trees m), Fix.Acc. (Fixed-point Accuracy), XGB.Acc. (XGB Accuracy). T_{fixed} and T_{xgb} are running time in seconds. Fixed-point training is slower than XGBoost due to the absence of low-level optimizations, but remains practical in absolute terms (30s on the largest datasets).

(data, h, m)	Fix.Acc.	XG.Acc.	T_{fixed}	T_{xgb}
(CR, 4, 50)	0.8204	0.8208	2.70	0.13
(CR, 4, 100)	0.8200	0.8185	5.37	0.19
(CR, 5, 50)	0.8216	0.8191	3.41	0.15
(CR, 5, 100)	0.8191	0.8171	6.83	0.22
(CO, 4, 50)	0.8216	0.8216	13.64	0.47
(CO, 4, 100)	0.8415	0.8410	27.05	0.73
(CO, 5, 50)	0.8382	0.8414	16.52	0.50
(CO, 5, 100)	0.8653	0.8633	33.11	0.77

Table 2: Running time in LAN (Unit: minutes), communication overhead (Unit: GB), and memory usage (Unit: GB). Latency refers to the additional delay emulated on top of LAN.

Bandwidth	1Gbps			Comm.	Mem.
Latency	+0	+10	+20		
(BR, 50)	9.28	9.76	9.72	33.19	0.40
(BR, 100)	18.74	19.27	19.49	66.37	0.41
(CR, 50)	13.44	13.55	13.84	44.25	1.17
(CR, 100)	26.58	27.08	27.50	88.05	1.18
(CO, 50)	38.29	39.19	40.18	120.56	2.89
(CO, 100)	75.60	77.70	79.66	239.36	2.93

binary classification), **Coverttype 50k** ($n = 50000, d = 54$) and **Coverttype 100k** ($n = 100,000, d = 54$), and **Adult** ($n = 45222, d = 104$). We abbreviate the first three to **BR**, **CR**, **CO**.

Due to space constraints we are presenting two of our experiments in the main body in Table 1. The rest can be found in Appendix G.1 as well as the detailed performance discussion in Appendix G.2.

Accuracy parity. Across all five benchmarks: Credit Default and Coverttype-100k in Table 1, and Breast Cancer in Table 5, Coverttype-50k in Table 6, Adult in Table 7 and the tested hyperparameter grid, our fixed-point training *tracks* floating-point XGBoost to within statistical error (the last three can be found in Appendix G.1).

Performance. Our fixed-point implementation is slower than XGBoost, which is a highly optimized C++ library leveraging vectorization, cache tuning, and parallelism. A rewrite in a low level language with all the relevant optimizations should substantially reduce constant factors.

4.2 Zero Knowledge XGBoost-FP

Implementation and Setup. We implement the ZK XGBoost-FP² in C++ over the VOLE-ZK framework [YSWW21a] based on EMP-toolkits [WMK16] and the ZK-RAM from [YH24]. All experiments only employ 1 thread and report end-to-end running time. We conduct our experiments on two AWS EC2 m5.2xlarge instances located in the same region. Each is equipped with 8 vCPUs and 32GB RAM. To simulate various network condition, we use the Linux tc tool to configure the bandwidth and latency.

Running Time in Different Networks. We measure the performance of ZK XGBoost-FP in local and wide area networks, and present the results in Table 2 and 3. Throughout all experiments, the running time is not impacted by the emulated latency due to low round complexity. The results

² Accessible at <https://anonymous.4open.science/r/zk-xgboost-37DA>

Table 3: Running time in WAN (Unit: minutes). Latency refers to the additional delay emulated on top of LAN.

Latency	+30ms			+60ms		
Band.	800	400	200	800	400	200
(BR, 50)	10.29	15.64	26.38	11.74	16.19	27.04
(BR, 100)	20.54	31.2	52.77	23.25	32.42	54.04
(CR, 50)	14.72	21.45	35.57	16.61	22.26	36.45
(CR, 100)	29.9	42.39	70.85	33.01	44.29	72.48

Table 4: Microbenchmark on the Credit Default dataset (Unit: seconds).

	Bin	Logit	Leaf	Histogram	Inference
LAN	7.21	0.0026	5.3	8.1	2
WAN	13.45	0.0035	8.23	14.23	3.8

in WAN demonstrate the impact of restricted bandwidth on the performance: The time increases by around 50% and 65% when the bandwidth is decreased from 800Mbps to 400Mbps, then to 200Mbps. Table 3 doesn’t record CR-(50,100) for their excessive running time. However, even the largest instance CR-100 can still finish in 375 minutes in the worse network (200Mbps,+60ms).

The communication overhead listed in Table 2 depends on the dataset dimension (n, d) and number of trees. The memory overhead ranging from 0.4 to 2.93 GB is acceptable by most of devices.

Microbenchmark. We conduct a microbenchmark to demonstrate the time usage of binning, computing base logit, building leaf histogram, building internal nodes histogram, and inference. Table 4 shows the running time for these procedures. Note that the last three columns are the time validating per tree. Rebuilding the histogram on the tree and validating the splits consume the majority of proof time, due to the excessive invocation of RAM and division.

zkPoT for Random Forest. Our scheme can be adapted to prove random forest training as shown in Appendix F. Compared to Sparrow [PP24], our protocol has stronger security guarantees: it hides tree topology by filling a full binary tree with dummy nodes for better privacy, and enforces more rigorous consistency checks for nonlinear proofs (see Section 3.3). These additional guarantees lead to a larger circuit and, consequently, higher proof complexity. Nevertheless, our experiments show that the resulting ZKP remains only slightly slower than [PP24] in this setting. More details are presented in Appendix G.3.

5 Conclusion

We introduced the first zero-knowledge proof of training (zkPoT) for gradient boosted decision trees, realized concretely for XGBoost. Our approach reformulates XGBoost training as a deterministic, fixed-point algorithm with explicitly defined numerical semantics, and provides a cryptographic certification procedure that proves correct execution of the training process. We instantiate this certification using efficient VOLE-based zero-knowledge protocols and demonstrate that zkPoT for XGBoost is practical on standard datasets.

Beyond XGBoost, the certification approach can be generalized to other gradient-boosted decision tree systems. The zkPoT functionality can be extended to prove the quantitative fairness of model training on biased dataset, e.g., [RKVE20, SWF⁺23]. Besides, it would be interesting to realize the proposed XGBoost-FP in non-interactive succinct proof system for fast verification and public verifiability, while maintaining a proper prover cost.

Acknowledgments

We would like to thank Daniel Escudero for useful discussions during the early stages of this project. Nikolas Melissaris is supported by ERC grant OBELiSC (101115790).

Disclaimer

This paper was prepared in part for information purposes by the Artificial Intelligence Research group of JPMorgan Chase & Co and its affiliates (“JP Morgan”), and is not a product of the Research Department of JP Morgan. JP Morgan makes no representation and warranty whatsoever and disclaims all liability, for the completeness, accuracy or reliability of the information contained herein. This document is not intended as investment research or investment advice, or a recommendation, offer or solicitation for the purchase or sale of any security, financial instrument, financial product or service, or to be used in any way for evaluating the merits of participating in any transaction, and shall not constitute a solicitation under any jurisdiction or to any person, if such solicitation under such jurisdiction or to such person would be unlawful.

References

- [APKP24] K. Abbaszadeh, C. Pappas, J. Katz, and D. Papadopoulos. Zero-knowledge proofs of training for deep neural networks. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, pp. 4316–4330. ACM, 2024.
- [BBMH⁺21] C. Baum, L. Braun, A. Munch-Hansen, B. Razet, and P. Scholl. Appenzeller to brief: Efficient zero-knowledge proofs for mixed-mode arithmetic and Z2k. In *ACM CCS 2021*, pp. 192–211. ACM Press, 2021.
- [BBMHS22] C. Baum, L. Braun, A. Munch-Hansen, and P. Scholl. Moz $\mathbb{Z}_{2^k}$ arella: Efficient vector-OLE and zero-knowledge proofs over $\mathbb{Z}_{2^k}$. In *CRYPTO 2022, Part IV*, vol. 13510 of *LNCS*, pp. 329–358. Springer, Heidelberg, 2022.
- [BCG⁺13] E. Ben-Sasson, A. Chiesa, D. Genkin, E. Tromer, and M. Virza. Snarks for C: verifying program executions succinctly and in zero knowledge. In *Advances in Cryptology - CRYPTO 2013 - 33rd Annual Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2013. Proceedings, Part II*, vol. 8043 of *Lecture Notes in Computer Science*, pp. 90–108. Springer, 2013.
- [BGH19] S. Bowe, J. Grigg, and D. Hopwood. Recursive proof composition without a trusted setup. Cryptology ePrint Archive, Paper 2019/1021, 2019.
- [BMRS21] C. Baum, A. J. Malozemoff, M. B. Rosen, and P. Scholl. Mac’n’cheese: Zero-knowledge proofs for boolean and arithmetic circuits with nested disjunctions. In *CRYPTO 2021, Part IV*, vol. 12828 of *LNCS*, pp. 92–122, Virtual Event, 2021. Springer, Heidelberg.
- [CFQ19] M. Campanelli, D. Fiore, and A. Querol. Legosnark: Modular design and composition of succinct zero-knowledge proofs. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pp. 2075–2092, 2019.
- [CG16] T. Chen and C. Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD ’16*, p. 785–794, New York, NY, USA, 2016. Association for Computing Machinery.
- [CWSK24] B. Chen, S. Waiwitlikhit, I. Stoica, and D. Kang. ZKML: an optimizing system for ML inference in zero-knowledge proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22-25, 2024*, pp. 560–574. ACM, 2024.

- [DIO21] S. Dittmer, Y. Ishai, and R. Ostrovsky. Line-Point Zero Knowledge and Its Applications. In *2nd Conference on Information-Theoretic Cryptography (ITC 2021)*, vol. 199 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pp. 5:1–5:24, Dagstuhl, Germany, 2021. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [FKL⁺21] N. Franzese, J. Katz, S. Lu, R. Ostrovsky, X. Wang, and C. Weng. Constant-overhead zero-knowledge for RAM programs. In *ACM CCS 2021*, pp. 178–191. ACM Press, 2021.
- [FQZ⁺21] B. Feng, L. Qin, Z. Zhang, Y. Ding, and S. Chu. ZEN: An optimizing compiler for verifiable, zero-knowledge neural network inferences. Cryptology ePrint Archive, Paper 2021/087, 2021.
- [GGG17] Z. Ghodsi, T. Gu, and S. Garg. Safetynets: Verifiable execution of deep neural networks on an untrusted cloud. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pp. 4672–4681, 2017.
- [GGJ⁺23] S. Garg, A. Goel, S. Jha, S. Mahloujifar, M. Mahmood, G. Policharla, and M. Wang. Experimenting with zero-knowledge proofs of training. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023, Copenhagen, Denmark, November 26-30, 2023*, pp. 1880–1894. ACM, 2023.
- [GJJZ22] S. Garg, A. Jain, Z. Jin, and Y. Zhang. Succinct zero knowledge for floating point computations. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS ’22*, p. 1203–1216, New York, NY, USA, 2022. Association for Computing Machinery.
- [GKR15] S. Goldwasser, Y. T. Kalai, and G. N. Rothblum. Delegating computation: Interactive proofs for muggles. *J. ACM*, 62(4):27:1–27:64, 2015.
- [GMR89] S. Goldwasser, S. Micali, and C. Rackoff. The knowledge complexity of interactive proof systems. *SIAM J. Comput.*, 18(1):186–208, 1989.
- [Gol01] O. Goldreich. *Foundations of Cryptography: Volume 1, Basic Tools*. Cambridge University Press, 2001.
- [GOV22] L. Grinsztajn, E. Oyallon, and G. Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022.
- [HCL⁺24] M. Hao, H. Chen, H. Li, C. Weng, Y. Zhang, H. Yang, and T. Zhang. Scalable zero-knowledge proofs for non-linear functions in machine learning. In *33rd USENIX Security Symposium (USENIX Security 24)*, pp. 3819–3836, 2024.
- [IKOS09] Y. Ishai, E. Kushilevitz, R. Ostrovsky, and A. Sahai. Zero-knowledge proofs from secure multiparty computation. *SIAM J. Comput.*, 39(3):1121–1152, 2009.
- [LF25] Y. Li and X. Fan. SUMMER: Recursive zero-knowledge proofs for scalable RNN training. Cryptology ePrint Archive, Paper 2025/1688, 2025.
- [LKKO24] S. Lee, H. Ko, J. Kim, and H. Oh. vcnn: Verifiable convolutional neural network based on zk-snarks. *IEEE Trans. Dependable Secur. Comput.*, 21(4):4254–4270, 2024.
- [LXZ21] T. Liu, X. Xie, and Y. Zhang. zkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *ACM CCS 2021*, pp. 2968–2985. ACM Press, 2021.
- [PHGR13] B. Parno, J. Howell, C. Gentry, and M. Raykova. Pinocchio: Nearly practical verifiable computation. In *2013 IEEE Symposium on Security and Privacy*, pp. 238–252. IEEE Computer Society Press, 2013.

- [PP24] C. Pappas and D. Papadopoulos. Sparrow: Space-efficient zkSNARK for data-parallel circuits and applications to zero-knowledge decision trees. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, pp. 3110–3124. ACM, 2024.
- [RKVE20] S. Ravichandran, D. Khurana, B. Venkatesh, and N. U. Edakunni. Fairxgboost: Fairness-aware classification in xgboost, 2020.
- [SBLZ25] H. Sun, T. Bai, J. Li, and H. Zhang. zkdl: Efficient zero-knowledge proofs of deep learning training. *IEEE Trans. Inf. Forensics Secur.*, 20:914–927, 2025.
- [SGL24] A. Shmuel, O. Glickman, and T. Lazebnik. A comprehensive benchmark of machine and deep learning across diverse tabular datasets. *CoRR*, abs/2408.14817, 2024.
- [SLY⁺25] Y. Sun, H. Liu, K. Yang, Y. Yu, X. Wang, and C. Weng. Committed vector oblivious linear evaluation and its applications. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, pp. 3635–3648, 2025.
- [SWF⁺23] A. S. Shamsabadi, S. C. Wyllie, N. Franzese, N. Dullerud, S. Gambs, N. Papernot, X. Wang, and A. Weller. Confidential-profft: Confidential proof of fair training of trees. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.
- [TGM⁺25] G. Tan, A. Gascón, S. Meiklejohn, M. Raykova, X. Wang, and N. Luo. Founding zero-knowledge proofs of training on optimum vicinity. *Cryptology ePrint Archive*, Paper 2025/053, 2025.
- [WMK16] X. Wang, A. J. Malozemoff, and J. Katz. EMP-toolkit: Efficient MultiParty computation toolkit. <https://github.com/emp-toolkit>, 2016.
- [WSS⁺24] S. Waiwitlikhit, I. Stoica, Y. Sun, T. Hashimoto, and D. Kang. Trustless audits without revealing data or models, 2024.
- [WYKW21] C. Weng, K. Yang, J. Katz, and X. Wang. Wolverine: Fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In *2021 IEEE Symposium on Security and Privacy*, pp. 1074–1091. IEEE Computer Society Press, 2021.
- [WYX⁺21a] C. Weng, K. Yang, X. Xie, J. Katz, and X. Wang. Mystique: Efficient conversions for {Zero-Knowledge} proofs with applications to machine learning. In *30th USENIX Security Symposium (USENIX Security 21)*, pp. 501–518, 2021.
- [WYX⁺21b] C. Weng, K. Yang, X. Xie, J. Katz, and X. Wang. Mystique: Efficient conversions for zero-knowledge proofs with applications to machine learning. In *USENIX Security 2021*, pp. 501–518. USENIX Association, 2021.
- [YH24] Y. Yang and D. Heath. Two shuffles make a {RAM}: Improved constant overhead zero knowledge {RAM}. In *33rd USENIX Security Symposium (USENIX Security 24)*, pp. 1435–1452, 2024.
- [YSWW21a] K. Yang, P. Sarkar, C. Weng, and X. Wang. Quicksilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pp. 2986–3001, 2021.
- [YSWW21b] K. Yang, P. Sarkar, C. Weng, and X. Wang. QuickSilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In *ACM CCS 2021*, pp. 2986–3001. ACM Press, 2021.
- [ZC24] H. Zhu and S. C.-K. Chau. Optimizing zero-knowledge proofs for verifiable decision trees. In *IEEE Symposium on Security and Privacy (SP) Poster Session*, 2024.

[ZFZS20] J. Zhang, Z. Fang, Y. Zhang, and D. Song. Zero knowledge proofs for decision tree predictions and accuracy. In *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*, pp. 2039–2053. ACM, 2020.

A Additional Preliminaries

A.1 Further Details of XGBoost

Theoretical Background

XGBoost is a scalable implementation of gradient boosted decision trees. Given a dataset $\mathcal{D}$ and a differentiable convex loss function $\ell : \mathbb{R} \times \mathcal{Y} \rightarrow \mathbb{R}$, the algorithm trains an ensemble of m regression trees $\mathcal{T} = \{T_k\}_{k=1}^m$ such that the final prediction is

$$p = \sigma(z) \quad z = z_0 - \sum_{k=1}^m \eta \cdot w_{k,i}$$

where σ is the sigmoid function, z is the final logit, $w_{k,i} = T_k(\mathbf{x}_i)$ is the output of tree T_k on input $\mathbf{x}_i$, $\eta \in (0, 1]$ is the learning rate, and z_0 is the initial prediction (logit). The training algorithm builds trees iteratively to minimize the regularized objective. At each boosting round k , the algorithm computes the first- and second-order derivatives of the loss:

$$g_{k,i} = \frac{\partial}{\partial p_{k-1}} \ell(p_{k-1}, y_i), \quad h_{k,i} = \frac{\partial^2}{\partial p_{k-1}^2} \ell(p_{k-1}, y_i),$$

where p_{k-1} is the prediction after $k-1$ rounds. These $(g_{k,i}, h_{k,i})$ values are called *gradients* and *Hessians*.

Split finding. For each candidate split (feature f and threshold t), the gain is computed as

$$\text{Gain}(f, t) = \frac{1}{2} \cdot \left(\frac{G_{\mathcal{L}}^2}{H_{\mathcal{L}} + \lambda} + \frac{G_{\mathcal{R}}^2}{H_{\mathcal{R}} + \lambda} - \frac{(G_{\mathcal{L}} + G_{\mathcal{R}})^2}{H_{\mathcal{L}} + H_{\mathcal{R}} + \lambda} \right) - \gamma,$$

where $G_{\mathcal{L}} = \sum_{i \in \mathcal{L}} g_i$, $H_{\mathcal{L}} = \sum_{i \in \mathcal{L}} h_i$, and similarly for the right child $\mathcal{R}$. Hyperparameters λ, γ control regularization and split pruning. The best split maximizes the gain.

Leaf weights. Once a tree is grown, each leaf l receiving samples $\mathcal{I}_l$ gets a weight

$$w_l = \frac{\sum_{i \in \mathcal{I}_l} g_i}{\sum_{i \in \mathcal{I}_l} h_i + \lambda}.$$

Objective. The ensemble minimizes the regularized objective

$$L_k = \sum_{i=1}^n [g_{k,i} T_k(\mathbf{x}_i) + \frac{1}{2} h_{k,i} T_k(\mathbf{x}_i)^2] + \Omega(T_k),$$

$$\Omega(T) = \gamma N + \frac{1}{2} \lambda \sum_l w_l^2,$$

where N is the number of leaves. Training iteratively adds trees until m rounds are completed.

Loss function. In this work, we use the cross-entropy loss $\ell(p, y) = -y \log(p) - (1-y) \log(1-p)$, where $p = \sigma(z) \in (0, 1)$ is the predicted probability. Then the gradients and Hessians wrt logit are

$$g = \frac{\partial \ell}{\partial z} = p - y, \quad h = \frac{\partial^2 \ell}{\partial z^2} = p(1-p).$$

XGBoost Inference

Computation of the prediction for a given data point $\mathbf{x}$ proceeds in three steps:

1. **Tree Evaluation:** For each tree $T_k \in \mathcal{T}$, compute the weight $w_k = T_k(\mathbf{x})$.

2. **Score Aggregation:** Compute the final score (or *logit*) z :

$$z = z_0 - \sum_{k=1}^m \eta \cdot w_k$$

where z_0 is the base score determined during training and η is the learning rate.

3. **Probability Transformation:** Transform the score z in the log-odds space via the sigmoid function σ to yield the final probability:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}} \quad (6)$$

XGBoost Training

To train a tree ensemble $\mathcal{T}$ on a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, XGBoost first initializes the base score z_0 , pre-bins features into B discrete bins, and then iteratively builds trees T_k for boosting rounds $k = 1, \dots, m$ by selecting splits and computing leaf weights to maximize the gain. Note that z_0 is also part of the trained model, so the final model $\mathcal{M}$ consists of $\mathcal{T}$ and z_0 . We elaborate on these steps below:

1. **Base Score Initialization.** It initializes the base score z_0 by taking the log-odds

$$z_0 = \sigma^{-1}(p) = \log \frac{p}{1-p} \quad (7)$$

where $p = \frac{1}{n} \sum_i y_i$ is the mean label probability.

2. **Pre-binning.** Each feature column is discretized into B equal-width bins, and a lookup table $\text{edges} \in \mathbb{R}^{d \times B}$ of left edges of bins is constructed. This *pre-binning* step replaces direct floating-point comparisons on raw feature values with integer bin indices, enabling efficient and deterministic split evaluation.
3. For each boosting round $k = 1, \dots, m$, it builds a new tree T_k with the following key steps.
 - (a) **Prediction and derivatives.** For every data point $i \in [n]$, the algorithm computes the current probability $p_i = \sigma(z_{k-1,i})$, the gradient $g_i = p_i - y_i$, and the Hessian $h_i = p_i(1-p_i)$.
 - (b) **Split search.** For each node, containing samples indexed by $\mathcal{I} \subseteq [n]$, the algorithm scans candidate pairs (f, b) of feature and bin threshold defining partitions $\mathcal{L}$ and $\mathcal{R}$ such that $\mathcal{L} \cup \mathcal{R} = \mathcal{I}$. Then the best split (f^*, b^*) is selected by maximizing the gain:

$$\text{gain} = \frac{1}{2} \cdot \left(\frac{G_{\mathcal{L}}^2}{H_{\mathcal{L}} + \lambda} + \frac{G_{\mathcal{R}}^2}{H_{\mathcal{R}} + \lambda} - \frac{G_{\mathcal{I}}^2}{H_{\mathcal{I}} + \lambda} \right) - \gamma,$$

where $G_{\mathcal{I}} = \sum_{i \in \mathcal{I}} g_i$, $H_{\mathcal{I}} = \sum_{i \in \mathcal{I}} h_i$, and similarly for the children $\mathcal{L}$ and $\mathcal{R}$. Hyperparameters λ, γ control regularization and split pruning.

- (c) **Leaf weight computation.** Once the tree structure is finalized, each leaf $l \in [N]$ is assigned the weight

$$w_l = \frac{G_{\mathcal{I}_l}}{H_{\mathcal{I}_l} + \lambda},$$

- (d) **Logit update.** Finally, for each data point $i \in [n]$, the logit is updated as

$$z_{k,i} = z_{k-1,i} - \eta \cdot T_k(\mathbf{x}_i).$$

A.2 Details of Commit-and-Prove Zero Knowledge Proofs

Functionality 1: $\mathcal{F}_{\text{CP}}[\mathcal{R}]$

The functionality $\mathcal{F}_{\text{CP}}$ interacts with three parties: a prover $\mathcal{P}$, a verifier $\mathcal{V}$, and an ideal adversary $\mathcal{S}$. It is parameterized by an NP relation $\mathcal{R}$.

Committing Phase Upon receiving (COMMIT, w) from $\mathcal{P}$:

- 1: Store $\mathbf{w}$ internally and send COMMIT to $\mathcal{S}$.
 - 2: Upon receiving (DO-COMMIT) from $\mathcal{S}$, send COMMIT-RECEIPT to $\mathcal{V}$.
- Proving Phase** Upon receiving (PROVE, $\mathbf{x}$, $\mathbf{w}$) from $\mathcal{P}$:
- 1: If $(\mathbf{x}, \mathbf{w}) \notin \mathcal{R}$, ignore the input.
 - 2: Else, send (PROVE, $\mathbf{x}$) to $\mathcal{S}$.
 - 3: Upon receiving DO-PROVE from $\mathcal{S}$, send (PROVE-RECEIPT, $\mathbf{x}$) to $\mathcal{V}$.

A.3 Zero-Knowledge Proof based on Vector Oblivious Linear Evaluation

A series of works construct interactive ZKPs based on the vector oblivious linear evaluation (VOLE) [DIO21, WYKW21, BMRS21, YSWW21b, WYX⁺21b, BBMH⁺21, BBMHS22, HCL⁺24]. They require a preprocessing (witness-independent) phase that generates a uniformly sampled VOLE correlation $\mathbf{w} = \mathbf{v} + \mathbf{u} \cdot \Delta$, from which $\mathcal{P}$ obtains $(\mathbf{u}, \mathbf{w})$ and $\mathcal{V}$ obtains $(\mathbf{v}, \Delta)$. Δ is denoted as the global key. In the proving (witness-dependent) phase, $\mathcal{P}$ and $\mathcal{V}$ transform the VOLE correlation into a commitment to the witness $\mathbf{x}$, i.e. $M[\mathbf{x}] = K[\mathbf{x}] + \mathbf{x} \cdot \Delta$. $M[\mathbf{x}]$ is denoted as the message authentication code and $K[\mathbf{x}]$ is denoted as the local key. We denote the commitment to $\mathbf{x}$ as $[\mathbf{x}]$.

Proving the linear relations is *free* over VOLE commitments since they are linearly homomorphic. I.e., the commitment to $[z] = [x] + [y]$ can be generated by defining $M[z] = M[x] + M[y]$ and $K[z] = K[x] + K[y]$. To prove the multiplicative relation $[z] = [x] \cdot [y]$, previous works [DIO21, YSWW21b] observe that

$$\begin{aligned} K[x] \cdot K[y] + K[z] \cdot \Delta \\ &= M[x] \cdot M[y] - (x \cdot M[y] + y \cdot M[x] - M[z]) \cdot \Delta \\ &\quad + (x \cdot y - z) \cdot \Delta^2 \end{aligned}$$

Note that $(x \cdot y - z) \cdot \Delta^2$ is canceled if $\mathcal{P}$ honestly commits to these values. Define $B = K[x] \cdot K[y] + K[z] \cdot \Delta$, $A_0 = M[x] \cdot M[y]$ and $A_1 = x \cdot M[y] + y \cdot M[x] - M[z]$, $\mathcal{P}$ simply proves that it holds a valid pair of (A_0, A_1) that satisfies $B = A_0 - A_1 \cdot \Delta$. This can be done by a mask-and-open. Furthermore, a large number of multiplicative relations can be proven in a batch by random linear combination.

For an arithmetic circuit of size C , the total communication overhead for the preprocessing phase is $O(\text{polylog} C)$ with 2 rounds and the online proving phase is $O(C)$ with 1 round. Notably, the end-to-end round complexity is 2 and its online phase can be made non-interactive.

B Additional Related Work

The field of zero-knowledge machine learning has evolved through two main phases: inference verification and training verification.

Foundational verifiable ML. Early work established the feasibility of verifiable neural network execution. [GGG17] with SafetyNets introduced verifiable execution of deep neural networks using interactive proofs based on the GKR protocol ([GKR15], representing DNNs as arithmetic circuits for verification with practical performance for networks with millions of parameters, though without zero-knowledge privacy).

Zero-knowledge inference verification. Recent systems have made ZK inference practical for large neural networks. [LKKO24] with vCNN presented the first practical zero-knowledge proof system for CNN inference using zk-SNARKs ([BCG⁺13]), developing circuit-friendly representations of convolution and pooling operations with 8500times speedup over naive approaches. [FQZ⁺21] with ZEN proposed zero-knowledge neural network inference using Quadratic Arithmetic Program-based zk-SNARKs with optimizations for ReLU activations. Building on these foundations, [CWSK24] in ZKML developed a compiler framework that automatically translates TensorFlow models into Halo2 zk-SNARK circuits ([BGH19]), achieving orders-of-magnitude reductions in proof generation time for large models like ResNet-18 and GPT-2 through specialized “gadgets” for activation functions and circuit layout optimization.

Decision tree inference verification. [ZFZS20] formalized the first zero-knowledge proof protocols specifically for decision tree inference and accuracy testing, enabling model owners to prove tree

predictions or accuracy without revealing structure or data. Their implementation demonstrates practical performance, proving accuracy of a 1,029-node tree on 5,000 test samples in about 250 seconds with 287 KB proof size. [ZC24] improved upon this work by replacing hash-based commitments with polynomial commitments, achieving over $50\times$ reduction in multiplication gates through a “modular” approach that decouples tree structure from parameters.

Proof of training Training verification is fundamentally more challenging than inference due to the iterative, data-intensive nature of learning. [GGJ⁺23] coined the term “zero-knowledge proofs of training” and provided the first rigorous security definitions, demonstrating feasibility with a zkPoT protocol for logistic regression using MPC-in-the-head ([IKOS09]) techniques combined with zk-SNARKs. For deep neural networks, [SBLZ25] presented zkDL, introducing zkReLU (SNARK-friendly ReLU gradient proofs) and FAC4DNN (aggregated circuit design), achieving proof generation for an 8-layer DNN (10M parameters, batch size 64) in under 1 second per batch. [APKP24] built Kaizen, a complete zkPoT system using Halo2 that proves the entire training process including weight updates while maintaining privacy of both data and model.

Decision tree fairness verification. Shamsabadi et al., [SWF⁺23] introduced Confidential-PROFIT, which proves decision trees were trained under certain fairness constraints. Their approach differs from traditional training verification; instead of proving each training step, they designed a certified training algorithm where information gain on sensitive attributes is bounded, outputting both an optimal tree and a zero-knowledge proof certifying fair training without revealing data or model parameters.

Why fixed-point arithmetic? Garg et al., [GJJZ22] showed that zero-knowledge proofs can certify IEEE-754 floating-point execution by explicitly modeling mantissa alignment, exponent comparison, normalization, and rounding at the bit level. However, each floating-point addition or multiplication expands into hundreds to thousands of Boolean or arithmetic constraints, due to bit decomposition, conditional logic, and normalization circuitry. As a result, the cost of proving a single floating-point operation is orders of magnitude larger than that of a fixed-point addition or multiplication. This overhead compounds in iterative learning workloads, where training involves millions of arithmetic operations across many boosting rounds. In contrast, our fixed-point formulation uses native field arithmetic with explicit truncation, yielding constant-overhead arithmetic operations and enabling scalable certification of gradient-boosted training.

C Details of Fixed-Point XGBoost Training

In this section, we provide further details of our fixed-point XGBoost training algorithm, including the complete training procedure (Procedure 1) and numerical safeguards (Section C.1). The notations and data structures used here are consistent with those in Section 2.1.

C.1 Numerical Safeguards in Fixed-Point XGBoost

To ensure that fixed-point XGBoost training admits a sound and reproducible zero-knowledge certification, we incorporate a set of numerical safeguards. Here we make them explicit and they are enforced as part of the certified training relation.

Binary label validation. We enforce that each training label satisfies $y_i \in \{0, 1\}$ by checking $y_i \cdot (1 - y_i) = 0$. This prevents malformed inputs that could invalidate gradient and Hessian computations.

Clipping of probabilities and logits. When computing probabilities $p = \sigma(z)$ and their inverse logits $z = \log \frac{p}{1-p}$, we apply explicit clipping. Probabilities are restricted to $[p_{\min}, 1 - p_{\min}]$ (with $p_{\min}$ typically around 10^{-6}), and logits are bounded to $[-L, L]$ where $L = \log \frac{1-p_{\min}}{p_{\min}}$. These clamps serve two roles: (i) they prevent numerical instabilities such as division by zero or $\log(0)$, which would otherwise cause divergence between the implementation and the certificate, and (ii) they align the forward and inverse mappings so that round-trips $\text{SIGMOIDFP}(\text{LOGITFROMPROBFP}(p))$ remain consistent under discretization.

Clipping of leaf weights. Each leaf weight is computed as $w = -G/(H + \lambda)$, but then passed through $\text{CLIPFP}(w, -1, 1)$. This a stability safeguard which avoids extreme updates when H is very small and is also part of the certified relation: the verifier explicitly checks that the prover has applied the same clamp. Without it, it wouldn’t be necessary that the certificate would match the training procedure.

Pre-binning of features. Features are first mapped into B equal-width bins, and the training algorithm works only on these integer bin indices. This removes floating-point comparisons on raw feature values from the relation and ensures that every branch decision in the tree is discrete making it easily verifiable in zero-knowledge.

Division safety checks. For every fixed-point division $z = \lfloor x/y \rfloor$, the protocol enforces auxiliary range constraints on x , y , z , and the remainder to prevent wrap-around modulo the field. In particular, denominators and quotients are required to lie below explicit bounds so that $x = z \cdot y + r$ holds without overflow.

Gain and histogram bounds. Aggregated gradient and Hessian values used in gain computation are range-checked to ensure that squaring and division operations do not overflow the field. These bounds are sufficient to guarantee correctness of split comparisons in the zero-knowledge proof.

Deterministic handling of terminal nodes. When no split yields positive gain, a deterministic terminal split is enforced. This avoids ambiguity in tree structure and ensures that training and certification follow identical control flow.

Procedure 1: XGBoost-FP

Parameters: trees m , tree height h , learning rate η , regularizers λ, γ , #bins B

Input: Training data $D = \{\mathbf{x}_i, y_i\}_{i=1}^n$.

Output: Sequence of trees $T_1, \dots, T_m$ and base logit z_0 .

TrainXGB(D):

```

1: for  $i = 1, \dots, n$ : assert  $y_i \in \{0, 1\}$ 
2:  $p \leftarrow (\sum_i y_i)/n$ 
3:  $\bar{p} \leftarrow \text{ClipFP}(p, p_{\min}, p_{\max})$ 
4:  $z_0 \leftarrow \text{LogitFromProbFP}(\bar{p}) \parallel z_0 \approx \log(\bar{p}/(1 - \bar{p}))$ 
5: Let  $z_{0,i} = z_0$  for all  $i \in [n]$ 
6:  $(\text{edges}, \text{binID}) \leftarrow \text{PreBinFP}(\mathbf{x}, B)$ 
7: for  $k \in [m]$  do
8:   for  $i \in [n]$  do
9:      $p_i \leftarrow \text{SigmoidWideFP}(z_{k-1,i}) \parallel p_i \approx \frac{1}{1 + \exp(-z_{k-1,i})}$ 
10:     $g_i \leftarrow p_i - y_i$ 
11:     $h_i \leftarrow p_i(1 - p_i)$ 
12:    $T_k \leftarrow (\mathbf{f}_k, \mathbf{t}_k, \mathbf{w}_k) = (\mathbf{0}, \mathbf{0}, \mathbf{0})$ 
13:    $\text{BuildTree}(T_k, 1, \{g_i\}_{i \in [n]}, \{h_i\}_{i \in [n]}, \text{binID}, \text{edges})$ 
14:   for  $i \in [n]$  do
15:     Let  $l_{k,i}$  be the leaf node where  $\mathbf{x}_i$  falls in  $T_k$ 
16:      $z_{k,i} = z_{k-1,i} - \eta \cdot w_{k,l_{k,i}}$ 
17: return  $\{T_k\}_{k \in [m]}$  and  $z_0$ 

```

BuildTree($T, \ell, \{g_i\}_{i \in \mathcal{I}}, \{h_i\}_{i \in \mathcal{I}}, \text{binID}, \text{edges}$):

```

1: if height( $T$ ) =  $h$  then
2:    $G \leftarrow \sum_{i \in \mathcal{I}} g_i$ ;  $H \leftarrow \sum_{i \in \mathcal{I}} h_i$ 
3:    $w' \leftarrow \frac{G}{H + \lambda}$ 
4:    $w \leftarrow \text{ClipFP}(w', -1, 1)$ 
5:    $T.w_{\ell-(N-1)} \leftarrow w$ 
6: else
7:    $(f^*, b^*, \text{gain}^*) \leftarrow \text{FindSplit}(\{g_i\}_{i \in \mathcal{I}}, \{h_i\}_{i \in \mathcal{I}}, \text{binID})$ 
8:   if  $\text{gain}^* \leq 0$  then // pruning via dummy splits; any sample will go right
9:      $b^* \leftarrow b_{\text{dum}} = 0$ 
10:     $f^* \leftarrow f_{\text{dum}}$ 
11:     $t^* \leftarrow \text{edges}[f^*][b^*]$ 
12:     $\mathcal{L} := \{i \in \mathcal{I} : \text{binID}[i][f^*] < b^*\}$ ;  $\mathcal{R} := \mathcal{I} \setminus \mathcal{L}$ 
13:     $\text{BuildTree}(T, 2\ell, \{g_i\}_{i \in \mathcal{L}}, \{h_i\}_{i \in \mathcal{L}}, \text{binID}, \text{edges})$ 
14:     $\text{BuildTree}(T, 2\ell + 1, \{g_i\}_{i \in \mathcal{R}}, \{h_i\}_{i \in \mathcal{R}}, \text{binID}, \text{edges})$ 
15:     $(T.f_\ell, T.t_\ell) \leftarrow (f^*, t^*)$ 

```

FindSplit($\{g_i\}_{i \in \mathcal{I}}, \{h_i\}_{i \in \mathcal{I}}, \text{binID}$):

```

1:  $f^* \leftarrow 0$ ;  $b^* \leftarrow 0$ ;  $\text{gain}^* \leftarrow -\infty$ 
2:  $G \leftarrow \sum_{i \in \mathcal{I}} g_i$ ;  $H \leftarrow \sum_{i \in \mathcal{I}} h_i$ 

```

```

3: for  $f \in [d]$  do
4:   for  $b \in [B]$  do
5:      $\mathcal{L} \leftarrow \{i \in \mathcal{I} : \text{binID}[i][f] < b\}$ 
6:      $G_{b,\mathcal{L}} \leftarrow \sum_{i \in \mathcal{L}} g_i; H_{b,\mathcal{L}} \leftarrow \sum_{i \in \mathcal{L}} h_i$ 
7:      $G_{b,\mathcal{R}} \leftarrow G - G_{b,\mathcal{L}}; H_{b,\mathcal{R}} \leftarrow H - H_{b,\mathcal{L}}$ 
8:
9:      $\text{gain} \leftarrow \frac{1}{2} \cdot \left( \frac{G_{b,\mathcal{L}}^2}{H_{b,\mathcal{L}} + \lambda} + \frac{G_{b,\mathcal{R}}^2}{H_{b,\mathcal{R}} + \lambda} - \frac{G^2}{H + \lambda} \right) - \gamma$ 
10:    if  $\text{gain}^* < \text{gain}$  then
11:       $\text{gain}^* \leftarrow \text{gain}; f^* \leftarrow f; b^* \leftarrow b$ 
12: return the best split  $(f^*, b^*, \text{gain}^*)$ 

```

D Details of Certification Algorithm for XGBoost Training

In this section, we provide the detailed procedure for the certification that verifies the correctness of XGBoost training. The procedure is summarized in Procedure 2. The algorithm takes as input a training data $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_n)$ with labels $\mathbf{y} = (y_1, \dots, y_n)$, a fitted tree ensemble $\mathcal{T} = \{T_k\}_{k \in [m]}$, where each tree T_k is represented by $(\mathbf{f}_k, \mathbf{t}_k, \mathbf{w}_k)$, and a base logit z_0 . It then verifies that $(\mathcal{T}, z_0)$ is the output of the fixed-point XGBoost training algorithm **TrainXGB** (see Section 3.1 and Appendix C) on input $(\mathbf{x}, \mathbf{y})$.

Claim 1. *The certificate algorithm **CertXGB** verifies the correctness of XGBoost training. That is, for any $(\mathbf{x}, \mathbf{y}), \mathcal{T} = \{T_k\}_{k \in [m]}, z_0$, $\text{CertXGB}(\mathbf{x}, \mathbf{y}, \mathcal{T}, z_0) = 1$ if and only if $(\mathcal{T}, z_0) = \text{TrainXGB}(\mathbf{x}, \mathbf{y})$.*

Proof. **(If)** If $(\mathcal{T}, z_0) = \text{TrainXGB}(\mathbf{x}, \mathbf{y})$, then all the assertions in the subprotocols clearly hold, and thus **CertXGB** returns 1.

(Only if) For fixed $(\mathbf{x}, \mathbf{y}, \mathcal{T}, z_0)$, suppose $(\mathcal{T} = (\mathbf{f}_k, \mathbf{t}_k, \mathbf{w}_k)_{k=1}^m, z_0) \neq (\mathcal{T}' = (\mathbf{f}'_k, \mathbf{t}'_k, \mathbf{w}'_k)_{k=1}^m, z'_0) = \text{TrainXGB}(\mathbf{x}, \mathbf{y})$. We show that $\text{CertXGB}(\mathbf{x}, \mathbf{y}, \mathcal{T}, z_0) = 0$.

If $z_0 \neq z'_0$: Since **ValidateLogit** computes the logit from $\mathbf{y}$ as in **TrainXGB**, **CertXGB** derives the same z'_0 from $\mathbf{y}$. Thus, it holds that $z_0 \neq z'_0$, and thus **CertXGB** returns 0.

If $T_1 \neq T'_1$ and $(\mathbf{f}_1, \mathbf{t}_1) \neq (\mathbf{f}'_1, \mathbf{t}'_1)$ (i.e., the first tree contains an incorrect split): Since **CertXGB** runs **InitHists**, which computes p_i, g_i , and h_i from $z_{i,0} = z_0$ for $i = 1, \dots, n$ as in **TrainXGB**, they are the same as those in **TrainXGB**. **InitHists** also aggregates them into root histograms from leaf histograms. Thus, every possible split and **gain** computed inside **ValidateSplits** for the root are the same as those in **FindSplit** of **TrainXGB**.

We first consider the case where $(f_{1,1}, t_{1,1}) \neq (f'_{1,1}, t'_{1,1})$, i.e., the split at the root node is incorrect. In **CertXGB**, it computes $(f_{1,1}^*, t_{1,1}^*)$ and the corresponding max **gain** from the root histograms, which are the same as those in **TrainXGB**. It runs **ValidateSplits**, which checks either of the following cases: (1) if gain^* is non-positive, then it checks whether $(f_{1,1}, t_{1,1})$ has dummies; (2) else, it checks whether $(f_{1,1}, t_{1,1}) = (f_{1,1}^*, t_{1,1}^*)$. In the former case, **TrainXGB** would also set dummies for $(f'_{1,1}, t'_{1,1})$ and thus the assertion by **ValidateSplits** fails. In the latter case, since **TrainXGB** chooses the optimal split $(f_{1,1}^*, t_{1,1}^*)$ which is different from $(f_{1,1}, t_{1,1})$, the assertion by **ValidateSplits** also fails. Note that **ValidateSplits** also checks that $(f_{1,1}^*, t_{1,1}^*)$ is the smallest among those achieving the max **gain**, which also holds since **TrainXGB** chooses the smallest one in case of ties.

Now, we consider the case where $(f_{1,1}, t_{1,1}) = (f'_{1,1}, t'_{1,1})$, i.e., the split at the root node is correct, but some other split in T_1 is incorrect. By **ValidateInference**, the samples reaching left and right child nodes of the root in **CertXGB** are the same as those in **TrainXGB**. Thus, the histograms at these child nodes computed by **InitHists** are also the same as those in **TrainXGB**. Then, we can apply the same argument as above to either of the child nodes where the split is incorrect, and show that assertion by **ValidateSplits** fails in one of the second-level nodes. Applying this argument recursively, **CertXGB** always detects the incorrect split in T_1 .

If $T_1 \neq T'_1$ and $(\mathbf{f}_1, \mathbf{t}_1) = (\mathbf{f}'_1, \mathbf{t}'_1)$ but $\mathbf{w}_1 \neq \mathbf{w}'_1$ (i.e., the first tree contains an incorrect leaf weight): Since the splits are correct, by **ValidateInference**, the samples reaching each leaf in **CertXGB** are the same as those in **TrainXGB**. Thus, the histograms at these leaves computed by **InitHists** are also the same as those in **TrainXGB**. Then, **ValidateLeafWeights** checks whether the leaf weights are correctly computed from these histograms as in **TrainXGB**, and thus the assertion fails for the incorrect leaf weight.

$T_1 = T'_1$ but for some $k \leq m$, $T_k \neq T'_k$ (i.e., one of the subsequent trees is incorrect): Since CertXGB correctly computes the updated logits $\mathbf{z}_1$ after training T_1 , which are the same as those in TrainXGB, we can apply the same arguments as above iteratively to T_k for $k = 2, \dots, m$ and show that CertXGB detects the incorrect tree.

Therefore, by induction, for each $k \in [m]$, T_k is correctly trained on $(\mathbf{x}, \mathbf{y})$ with the initial scores $\mathbf{z}_{k-1}$, and the updated scores $\mathbf{z}_k$ are correctly computed. This implies that $(\mathcal{T}, z_0) = \text{TrainXGB}(\mathbf{x}, \mathbf{y})$. $\square$

Procedure 2: CertXGB

Parameters: Number of points n , features d , trees m , bins B , leaves $N = 2^h$ at depth h , learning rate η , regularizers λ, γ .

Input: Fixed-point training data $\mathcal{D} = (\mathbf{x}, \mathbf{y})$, the fitted tree ensemble $\mathcal{T} = \{T_k\}_{k \in [m]}$, and base logit z_0 .

Output: 1 (accept) or 0 (reject).

```

1: for  $i = 1, \dots, n$ : assert  $y_i \in \{0, 1\}$ 
2:  $(\text{edges}, \text{binID}) \leftarrow \text{PreBinFP}(\mathbf{x}, B)$ 
3: assert  $\text{ValidateLogit}(\mathbf{y}, z_0) = 1$ 
4:  $\mathbf{z}_0 \leftarrow (z_{0,i})_{i=1}^n$  with  $z_{0,i} = z_0$ 
5: for each tree  $k \in [m]$  do // Initialize intermediate scores
6:   for each data point  $i \in [n]$  do
7:     compute the reached leaf  $l_{k,i} \in [N]$  by evaluating  $T_k$  on  $\mathbf{x}_i$ 
8:      $z_{k,i} \leftarrow z_{k-1,i} - \eta \cdot w_{k,l_{k,i}}$ 
9:    $\mathbf{z}_k \leftarrow (z_{k,i})_{i=1}^n$ 
10:   $\mathbf{l}_k \leftarrow (l_{k,i})_{i=1}^n$ 
11: for each tree  $k \in [m]$  do // Tree validation
12:    $(\mathbf{f}_k, \mathbf{t}_k, \mathbf{w}_k) \leftarrow T_k$ 
13:   assert  $\text{ValidateInference}(\mathbf{x}, \mathbf{f}_k, \mathbf{t}_k, \mathbf{l}_k) = 1$ 
14:    $(G_k, H_k) \leftarrow \text{InitHists}(\mathbf{z}_{k-1}, \mathbf{y}, \text{binID}, \mathbf{l}_k)$ 
15:   for each internal node  $\ell \in [N-1]$  do
16:     compute  $(f_{k,\ell}^*, b_{k,\ell}^*)$  such that gain derived from  $(G_k, H_k)$  is maximized
17:      $t_{k,\ell}^* \leftarrow \text{edges}[f_{k,\ell}^*][b_{k,\ell}^*]$ 
18:    $(\mathbf{f}_k^*, \mathbf{t}_k^*) \leftarrow ((f_{k,\ell}^*, t_{k,\ell}^*))_{\ell=1}^{N-1}$ 
19:   assert  $\text{ValidateLeafWeights}(G_k, H_k, \mathbf{w}_k) = 1$ 
20:   assert  $\text{ValidateSplits}(G_k, H_k, \mathbf{f}_k, \mathbf{t}_k, \mathbf{f}_k^*, \mathbf{t}_k^*, \text{edges}) = 1$ 
21: return 1 (accept)

```

Procedure 3: PreBinFP

Input: Fixed-point feature matrix $\mathbf{x} = (x_{i,j})_{i \in [n], j \in [d]}$, number of bins B

Output: $\text{edges} \in \mathbb{N}^d \times \mathbb{R}^{B+1}$, and $\text{binID} \in [B]^{n \times d}$

```

1: for  $f = 1$  to  $d$  do
2:    $c_{\min} \leftarrow \min_{i \in [n]} x_{i,f}$ ;  $c_{\max} \leftarrow \max_{i \in [n]} x_{i,f}$ 
3:    $\delta_f \leftarrow \frac{c_{\max} - c_{\min}}{B}$ 
4:   Define a lookup table  $\text{edges}[f] \leftarrow (c_{\min} + (b-1) \cdot \delta_f)_{b=1}^B$  consisting of left edges of  $B$  equal-width bins.
5:    $\text{edges}[f_{\text{dum}}][b_{\text{dum}}] \leftarrow t_{\text{dum}}$  // define dummy (small enough) edge for pruning, e.g.,  $t_{\text{dum}} = \text{DBL\_MIN}$ 
6:   for  $i = 1, \dots, n$ :

```

$$\text{binID}[i][f] \leftarrow \sum_{b=1}^B \mathbf{1}\{\text{edges}[f][b] \leq x_{i,f}\}$$

```

7: return  $(\text{edges}, \text{binID})$ 

```


Procedure 4: ValidateLogit

Parameters: $p_{\min}, p_{\max}$ for clipping probabilities
Input: Labels $\mathbf{y} = (y_i)_{i \in [n]}$ and base logit z_0
Output: Accept/Reject

- 1: $p \leftarrow \frac{1}{n} \sum_{i=1}^n y_i$
- 2: $p' \leftarrow \text{ClipFP}(p, p_{\min}, p_{\max})$
- 3: $u \leftarrow 2p' - 1$
- 4: $z'_0 \leftarrow 2 \cdot \left(u + \frac{u^3}{3} + \frac{u^5}{5} \right)$ // truncated atanh series with fixed-point ops
- 5: **assert** $z_0 = z'_0$

Procedure 5: ValidateInference

Input: Data points $\mathbf{x} = (x_{i,f})_{i \in [n], f \in [d]}$, node features $\mathbf{f}$, node thresholds $\mathbf{t}$, and leaf indices $\mathbf{l} = (l_i)_{i=1}^n$ reached by each $\mathbf{x}_i$
Output: Accept/Reject

- 1: Define a table of root-to-leaf paths $\text{path}[l] = (f_{l,j}, t_{l,j})_{j=1}^h$ for each $l \in [N]$
- 2: **for** $i = 1$ **to** n **do**
- 3: *(Path lookup)* Retrieve $\text{path}[l_i] = (f_{l_i,j}, t_{l_i,j})_{j=1}^h$.
- 4: *(Batched permutation)* Define a permutation $(\bar{x}_{i,1}, \dots, \bar{x}_{i,d})$ of $(x_{i,1}, \dots, x_{i,d})$ such that the first h entries align with the path features $(f_{l_i,1}, \dots, f_{l_i,h})$; **assert** permutation correctness.
- 5: *(Branch decisions)* For $j = 1, \dots, h$, set $a_j \leftarrow \mathbf{1}\{t_{l_i,j} \leq \bar{x}_{i,j}\}$ and **assert**:

$$l_i + 2^h - 1 = 2^h + \sum_{j=1}^h a_j \cdot 2^{h-j}.$$

Procedure 6: InitHists

Input: Scores $\mathbf{z} = (z_i)_{i=1}^n$, labels $\mathbf{y} = (y_i)_{i=1}^n$, binID, leaf indices $\mathbf{l} = (l_i)_{i=1}^n$
Output: Histograms (G, H)

- 1: **for** $f \in [d]$, $\ell \in [2N - 1]$, $b \in [B]$, initialize $G[f][\ell][b] \leftarrow 0$, $H[f][\ell][b] \leftarrow 0$.
- 2: **for** $i = 1$ **to** n **do**
- 3: $p_i \leftarrow \text{SigmoidWideFP}(z_i)$; $g_i \leftarrow p_i - y_i$; $h_i \leftarrow p_i \cdot (1 - p_i)$
- 4: **for** $f = 1$ **to** d **do**
- 5: $b \leftarrow \text{binID}[i][f]$; $\ell \leftarrow l_i + (N - 1)$
- 6: $G[f][\ell][b] \leftarrow G[f][\ell][b] + g_i$; $H[f][\ell][b] \leftarrow H[f][\ell][b] + h_i$
- 7: **for** $\ell = N - 1$ **down to** 1 **do** // propagate up to root
- 8: **for** $f = 1$ **to** d **do**
- 9: **for** $b = 1$ **to** B **do**
- 10: $G[f][\ell][b] \leftarrow G[f][2\ell][b] + G[f][2\ell + 1][b]$
- 11: $H[f][\ell][b] \leftarrow H[f][2\ell][b] + H[f][2\ell + 1][b]$
- 12: **return** (G, H)

Procedure 7: SigmoidWideFP

Input: Fixed-point logit
Output: $y \in \{0, \dots, \text{scale}\}$

- 1: **if** $z \leq -2$ **then**
- 2: **return** 0
- 3: **else if** $z \geq 2$ **then**
- 4: **return** 1
- 5: **else**
- 6: $y \leftarrow \frac{z + 2}{4}$ // integer arithmetic only
- 7: **return** y

Procedure 8: ValidateLeafWeights

Input: Leaf histograms (G, H) , weights $\mathbf{w} = (w_l)_{l=1}^N$

Output: Accept/Reject

- 1: **for** $l = 1$ **to** N **do**
- 2: $\ell \leftarrow l + (N - 1)$
- 3: $G^{\text{leaf}} \leftarrow \sum_{b=1}^B G[1][\ell][b]; \quad H^{\text{leaf}} \leftarrow \sum_{b=1}^B H[1][\ell][b]$ // use any feature f , as the sum is feature-independent
- 4: $w'_l \leftarrow \frac{G^{\text{leaf}}}{H^{\text{leaf}} + \lambda}$
- 5: **assert** $w_l = \text{ClipFP}(w'_l, -1, 1)$

Procedure 9: ValidateSplits

Input: Node histograms (G, H) , resulting splits $(\mathbf{f}, \mathbf{t}) = (f_\ell, t_\ell)_{\ell=1}^{N-1}$, indices $(\mathbf{f}^*, \mathbf{t}^*) = (f_\ell^*, t_\ell^*)_{\ell=1}^{N-1}$ leading to maximum gain, and **edges**. Let $e_0 = 0$.

Output: Accept/Reject

- 1: **for** $\ell = 1$ **to** $N - 1$ **do**
- 2: **for** $f = 1$ **to** d **do**
- 3: $G \leftarrow \sum_{b=1}^B G[f][\ell][b]; \quad H \leftarrow \sum_{b=1}^B H[f][\ell][b]$
- 4: $G_L \leftarrow 0, H_L \leftarrow 0$
- 5: **for** $b = 1$ **to** B **do**
- 6: $G_L \leftarrow G_L + G[f][\ell][b]; \quad H_L \leftarrow H_L + H[f][\ell][b]$
- 7: $G_R \leftarrow G - G_L; \quad H_R \leftarrow H - H_L$
- 8: $\text{gain}[f][b] \leftarrow \frac{1}{2} \cdot \left(\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right) - \gamma$ // fixed-point divisions
- 9: Retrieve b_ℓ^* s.t. $t_\ell^* = \text{edges}[f_\ell^*][b_\ell^*]$
- 10: Retrieve $\text{gain}^* \leftarrow \text{gain}[f_\ell^*][b_\ell^*]$
- 11: **assert** $(\text{gain}^* > \text{gain}[f][b]) \vee (\text{gain}^* = \text{gain}[f][b] \wedge f_\ell^* \leq f \wedge b_\ell^* \leq b)$ for all $f \in [d], b \in [B]$
- 12: // tie-breaker check
- 12: $e_\ell \leftarrow (\text{gain}^* \leq 0 \vee e_{\text{parent}})$ // check if the current node should be terminal, or the upper-level node is terminal
- 13: **assert** $(1 - e_\ell) \cdot (f_\ell = f_\ell^* \wedge t_\ell = t_\ell^*) \vee e_\ell \cdot (f_\ell = f_{\text{dum}} \wedge t_\ell = t_{\text{dum}}) = 1$ // if not dummy, use max gain split; else, use terminal split

Procedure 10: ClipFP

Input: x , lower bound a , upper bound b with $a \leq b$ represented as fixed-point numbers

Output: Clipped value y .

- 1: $y \leftarrow x$
- 2: **if** $y < a$ **then**
- 3: $y \leftarrow a$
- 4: **if** $y > b$ **then**
- 5: $y \leftarrow b$
- 6: **return** y

E Additional Details on ZK XGBoost-FP

In this section, we provide detailed explanation on the proof of comparison, division, and truncation, as well as additional components in proving XGBoost-FP shown in Figure 2.

E.1 Proof of Comparison

Our proof of comparison builds upon the idea of [HCL⁺24] to prove the comparison relation by bits-decomposition. It first converts the proof of comparison into a MSB proof $\mathbf{1}\{\llbracket x \rrbracket < \llbracket y \rrbracket\} = \text{MSB}(\llbracket x - y \rrbracket)$ due to the signed representation. This is true because $\text{MSB}(\llbracket x - y \rrbracket) = 1$ if $x - y < 0$ and $\text{MSB}(\llbracket x - y \rrbracket) = 0$ otherwise. We shift our focus into proving $\llbracket s \rrbracket = \text{MSB}(\llbracket z \rrbracket)$ for committed $(\llbracket z \rrbracket, \llbracket s \rrbracket)$.

To ensure the soundness of MSB proof, it requires a *one-bit gap* between the scale of encoded signed number and the underlying field: for a finite field $\mathbb{F}_p$, we require $x, y \in [0, \lfloor p/4 \rfloor] \cup [p - \lfloor p/4 \rfloor, p - 1]$. Otherwise, the subtraction may cause an overflow and lead to an incorrect result. Existing approaches all somehow require the $\mathcal{P}$ providing the rest bits of z and proving the correctness of bit-decomposition. **Bit-decomposition.** Assume that a $\mathcal{P}$ commits to $(\llbracket z \rrbracket, \llbracket s \rrbracket)$ and needs to prove that $\llbracket s \rrbracket = \text{MSB}(\llbracket z \rrbracket)$. Define $n, \mathbb{F}_p$ s.t. $\log |\mathbb{F}_p| \leq n$, a naive way to prove the sign is to apply a bit-decomposition as in [WYX⁺21b]. However, it incurs $O(n)$ communication cost to commit to all n bits $(z_0, \dots, z_{n-1})$ and need to prove an additional binary adder circuit. [HCL⁺24]’s approach only incurs $O(n/d)$ for an arbitrarily defined d by leveraging the proof of table lookup [FKL⁺21, YH24].

Without loss of generality, we assume (d, t) such that $n - 1 = d \cdot t$. The idea is to group every consecutive d bits of z into $(\tilde{z}_0, \dots, \tilde{z}_{t-1})$, in which $\tilde{z}_i = \sum_{j=0}^t 2^j \cdot z_{id+j}$. In this case, $\mathcal{P}$ only needs to commit to $t = (n - 1)/d + 1$ elements and prove that $z = 2^{n-1} \cdot s + \sum_{i=0}^{t-1} 2^{id} \cdot \tilde{z}_i$. Additionally, $\mathcal{P}$ needs to show that each $\tilde{z}_i \in [0, 2^d - 1]$ via a membership proof from a list $T = (0, 1, \dots, 2^d - 1)$, which can be instantiated by table lookup with only $O(1)$ cost per access [YH24].

Preventing Malicious Prover. A soundness issue with the above scheme is that a cheating $\mathcal{P}$ may commit to incorrect $(\tilde{z}_0, \dots, \tilde{z}_{t-1}, s)$ such that $z + p = 2^{n-1} \cdot s + \sum_{i=0}^{t-1} 2^{id} \cdot \tilde{z}_i$. For example, assume that $p = 2^{61} - 1$ and $z = 0$, a cheating $\mathcal{P}$ can commit to $s = 1$ and all $\tilde{z}_i = 2^d - 1$, and prove that $z = p \bmod p = 0$. Since $\bmod p$ happens implicitly in ZK operations, extra measures are needed to detect such wrap-around. [HCL⁺24] attempted to address this issue but its approach doubles the cost of secure comparison.

Our solution employs a Mersenne prime in the form of $p = 2^n - 1$, and rules out the only cheating case when $z = 0, s = 1$ and $\tilde{z}_i = 2^d - 1$ for all $i \in [0, t)$. By defining $\llbracket w \rrbracket = \mathbf{1}\{\llbracket z \rrbracket \neq 0\}$, we observe that

$$\text{MSB}(z) = \begin{cases} 0 & \text{if } z = 0, w = 0 \\ s & \text{if } z \neq 0, w = 1 \end{cases}. \text{ It implies that } (1 - w) \cdot s = 0 \text{ should always be true. Additionally,}$$

we employ the non-equality-zero check from [PHGR13] to prove $\llbracket w \rrbracket = \mathbf{1}\{\llbracket z \rrbracket \neq 0\}$. Our solution to address the soundness issue only requires committing 2 extra values and proving 3 multiplicative relations, which incurs less overhead than [HCL⁺24]. We defer the formal algorithm description to Figure 2 in Appendix E.

Cost Analysis. The proof of comparison requires committing to $3 + n/d$ witnesses, performing n/d lookup on a table of size 2^d , and proving 3 multiplicative relations. The proof for table lookup generally takes $O(1)$ per access. Its $O(2^d)$ setup cost will be amortized since the table is reused across the whole proof [YH24].

Protocol 2: ZKP Gadgets

Check Nonzero. On input $\llbracket x \rrbracket, \llbracket y \rrbracket$, prove that $\llbracket y \rrbracket = \mathbf{1}\{\llbracket x \rrbracket \neq 0\}$.

1. If $x = 0$, $\mathcal{P}$ commits to $\llbracket u \rrbracket$ such that $u = 0$. Otherwise, it commits to $u = x^{-1}$.
2. Prove that $\llbracket y \rrbracket - \llbracket u \rrbracket \cdot \llbracket x \rrbracket = 0$ and $(1 - \llbracket y \rrbracket) \cdot \llbracket x \rrbracket = 0$.

Comparison. On input $\llbracket x \rrbracket, \llbracket y \rrbracket, \llbracket s \rrbracket$, prove that $s = \mathbf{1}\{\llbracket x \rrbracket < \llbracket y \rrbracket\}$. Define parameters (d, t) such that $n - 1 = d \cdot t$. Construct a public lookup table $\mathcal{T} = (0, \dots, 2^d - 1)$.

1. Commits to $\llbracket z \rrbracket = \llbracket x \rrbracket - \llbracket y \rrbracket$ and $\llbracket s \rrbracket = \text{MSB}(\llbracket z \rrbracket)$. Decompose z into $(\llbracket \tilde{z}_0 \rrbracket, \dots, \llbracket \tilde{z}_{t-1} \rrbracket)$ such that $\tilde{z}_i = \sum_{j=0}^t 2^j \cdot z_{id+j}$.
2. Prove that for $i \in [0, t)$, $\tilde{z}_i = \mathcal{T}[\tilde{z}_i]$.
3. Prove that $\llbracket z \rrbracket = 2^{n-1} \cdot \llbracket s \rrbracket + \sum_{i=0}^{t-1} 2^{id} \cdot \llbracket \tilde{z}_i \rrbracket$.
4. Commit to $\llbracket w \rrbracket = \mathbf{1}\{\llbracket z \rrbracket \neq 0\}$ and prove its correctness using the above nonzero check. Prove that $(1 - \llbracket w \rrbracket) \cdot \llbracket s \rrbracket = 0$.

Division. On input $\llbracket x \rrbracket, \llbracket y \rrbracket, \llbracket z \rrbracket$, prove that $z = \lfloor x/y \rfloor$. Define m_q and m_d as the upper bound of the abstract value of the quotient and denominator.

1. $\mathcal{P}$ commits to abstract values $\llbracket \bar{x} \rrbracket, \llbracket \bar{y} \rrbracket, \llbracket \bar{z} \rrbracket$ and prove that $\bar{i} = (1 - 2 \cdot \text{MSB}(i)) \cdot i$ for $i \in \{x, y, z\}$.
2. $\mathcal{P}$ commits to the residual $\llbracket \bar{r} \rrbracket$ and prove that $\llbracket \bar{x} \rrbracket = \llbracket \bar{y} \rrbracket \cdot \llbracket \bar{z} \rrbracket + \llbracket \bar{r} \rrbracket$.
3. Prove that $0 \leq \llbracket \bar{r} \rrbracket < \llbracket \bar{y} \rrbracket$, $\llbracket \bar{y} \rrbracket \leq m_d$ and $\llbracket \bar{z} \rrbracket < m_q$.
4. Prove that $\text{MSB}(z) = \text{MSB}(x) \oplus \text{MSB}(y)$ using the fact that for any binary values α, β , $\alpha \oplus \beta = \alpha + \beta - 2\alpha \cdot \beta$.

Truncation. On input $\llbracket x \rrbracket, \llbracket z \rrbracket, f$, prove that $z = \lfloor x/2^f \rfloor$. The proof is the same as the division except that $y = 2^f$ is public so that its range check is avoided. Additionally, z is bounded by $m_q = \lfloor p/2^{f+1} \rfloor$. If p is a Mersenne prime, the check of $0 \leq \llbracket r \rrbracket < 2^f$ and $z < m_q$ can both be done by bit-decomposition proofs.

E.2 Proof of Division and Truncation

Division. Proof of division takes inputs $(\llbracket x \rrbracket, \llbracket y \rrbracket, \llbracket z \rrbracket)$ and proves that $z = \lfloor x/y \rfloor$. Note that this statement disallows directly proving it by $x = z \cdot y$ because of the floor operation. To simplify the problem, we assume that both (x, y) represent positive values. To generalize it to arbitrary inputs, we can first prove their absolute values and then determine the sign of z based on the signs of x, y . It can also be trivially generalized for fixed-point representations by left-shift x before the division (assume that the left shift does not overflow).

In our XGBoost-FP, the numerators and denominators may range from 0 to nearly $p/2^f$. Previous proofs of division are not suitable since they only work for bounded small values [LXZ21, PP24]. Another approach proves the floating-point division but requires thousands of constraints [WYX⁺21a].

To prove the relation, our approach leverages the existence of a residue $r \in [0, y)$ such that $x = z \cdot y + r$. Additionally, two range checks are required to maintain the soundness: $r \in [0, y)$, and $z \in [0, \lfloor p/y \rfloor]$. The first check ensures r is a residue. The second check is needed to prevent the similar wrap-around issue happened in the proof of MSB. Namely, there could be many possible pairs of (z', r') such that $x = z' \cdot y + r' \bmod p$ is z' . The proof should ensure a z that satisfies $x = z \cdot y + r$ without modulo p .

In our work, we define upper bounds for the positive denominator y and quotient z , denoted as m_d and m_q , such that $m_d \cdot m_q < p/2$. $\mathcal{P}$ proves that $\llbracket y \rrbracket < m_d$, $\llbracket r \rrbracket < \llbracket y \rrbracket$ and $\llbracket z \rrbracket < m_q$. This is not necessarily sufficient for general proof of division, but is sufficient for our XGBoost-FP since with a relatively large p , these values are expected to lie within a reasonable range.

Truncation. The truncation is a special type of division with public and positive denominators $y = 2^f$. It follows similar ideas as in the above proof of division to prove the range of r and z . However, its proof is much simpler because f is usually small so that the range check of $r \in [0, 2^f)$ can be implemented by a table lookup. Also, the Mersenne prime $p = 2^n - 1$ results in $\lfloor (p-1)/2^f \rfloor = 2^{n-f} - 1$, which also enables a range check of $z \in [0, 2^{n-f} - 1]$ to be resolved by our bit-decomposition proof.

We defer the formal algorithm description for division and truncation to Figure 2 in Appendix E.

Cost Analysis. The division proof invokes 7 comparison/MSB and proves 4 multiplicative relation. The truncation is simplified to 2 MSB proof, 1 bit-decomposition proof, 1 table lookup, and 2 multiplications. The concrete overhead depends on the underlying proof system and the construction of lookup table.

E.3 Additional Components

Input validation. Our model training requires the global truth $\mathbf{y}$ to be binary values. Hence, a check is performed on witnesses to ensure that $y_i \cdot (1 - y_i) = 0$ for $i \in [n]$. Our protocol also supports arbitrary input validation on input dataset $\mathbf{x}$ depending on the dataset type.

Global Range Checks. In principle, the proof of numerical computation loses soundness whenever the overflow occurs. This implies that a range check is needed for every intermediate values that are committed during the proof. This can be realized by the proof of comparison mentioned above. To reduce the overhead for range checks, we examine the XGBoost proof and find out that most of intermediate values are already bounded because of the checks that we applied in the proof of comparison, division, and truncation. Additional range checks are only required in a few places.

Essentially, the range check is required when validating the leaf weights and the splits of internal nodes. The former computes $w \leftarrow G/(H + \lambda)$. To respect the fixed-point representation, the proof in fact validates $(G \cdot \text{scale})/(H + \lambda)$ so that the result w is lifted by scale . Hence, the value $|G|$ should be restricted in $[0, p/(2 \cdot \text{scale})]$. Similarly, we compute $G^2/(H + \lambda)$ when validating the split. Since G^2 is already lifted by scale^2 , it can be directly fed into the proof of division without a truncation. However, we still need to validate that $G \in [-\sqrt{p/2}, \sqrt{p/2})$ to prevent the overflow when proving G^2 .

In some situations, the checks can be avoided if the input to specific operations are already bounded. When validating base logit z'_0 , the approximation of atanh is guaranteed not to overflow since $p \leftarrow \sum_{i=1}^n y_i/n < 1$, due to y_i being binary values. In validating the hessian $h_i \leftarrow p_i \cdot (1 - p_i)$, the overflow is also not needed because p_i is an output of the sigmoid function.

Proof of RAM Access. In the proof of XGBoost, proving the construction of leaf-layer histograms requires obviously adding g_i and h_i to the buckets indexed by their bin ID. This operation implies the read and write RAM access and we initialize it by ZK-RAM from [YH24]. Its per-access overhead is $O(1)$ field elements so the total overhead is linear to the number of trees, number of nodes on the tree, and number of buckets per node.

Public Commitment and Proof of Opening. Our framework also supports the commit-and-prove ZKP such that $\mathcal{P}$ can first publicly commit to the dataset $\mathcal{D} = (\mathbf{x}, \mathbf{y})$, model parameters T_k , and other auxiliary information, and later prove their consistency in ZKP. The public commitment can be transmitted to a public bulletin board or corresponding verifiers anytime before ZKP. This can be realized by [WYX⁺21b, SLY⁺25, CFQ19].

E.4 Security Analysis

We provide the security analysis for Theorem 1. We assume that $\mathcal{F}_{\text{RAMZK}}$ is instantiated by [YH24, YSWW21a] and $\mathcal{F}_{\text{CVOLE}}$ is instantiated by [SLY⁺25]. The secure realization of these functionalities can be referred to the cited works. We show Functionalities $\mathcal{F}_{\text{CVOLE}}$ and $\mathcal{F}_{\text{RAMZK}}$ below and reinstate Theorem 1.

Functionality 2: $\mathcal{F}_{\text{CVOLE}}$

The functionality interacts with three parties: a prover $\mathcal{P}$, a verifier $\mathcal{V}$, and an ideal adversary $\mathcal{S}$. It is parameterized by a secret $\mathbf{x}$ and its commitment $\text{com}_{\mathbf{x}}$.

1. Upon receiving `init` from both parties, and additional input $\mathbf{x}$ and decommitment message $\text{decom}_{\mathbf{x}}$ from $\mathcal{P}$, check whether $\text{com}_{\mathbf{x}}$ correctly decommits to $\mathbf{x}$.
2. Upon receiving `cvole` from both parties, sample a VOLE correlation $\mathbf{m} = \mathbf{k} + \mathbf{x} \cdot \Delta$. Send $(\mathbf{m})$ to $\mathcal{P}$ and $(\mathbf{k}, \Delta)$ to $\mathcal{V}$. If $\mathcal{P}$ is corrupted, receive $(\mathbf{m})$ from $\mathcal{S}$ and recompute $\mathbf{k} = \mathbf{m} - \mathbf{x} \cdot \Delta$. If $\mathcal{V}$ is corrupted, receive $(\mathbf{k}, \Delta)$ from $\mathcal{V}$ and recompute $\mathbf{m} = \mathbf{k} + \mathbf{x} \cdot \Delta$. Send these values to corresponding parties.

Functionality 3: $\mathcal{F}_{\text{RAMZK}}$

The functionality interacts with three parties: a prover $\mathcal{P}$, a verifier $\mathcal{V}$, and an ideal adversary $\mathcal{S}$. It takes input a circuit C that consists of arithmetic components and read-only or read-write RAM components. Upon receiving $(\text{prove}, C, \mathbf{x}, \mathbf{w})$ from $\mathcal{P}$ and $(\text{verify}, C, \mathbf{x})$, it verifies $C(\mathbf{x}, \mathbf{w})$ by traversing the circuit by topological order and check the relations.

1. For arithmetic components, it verifies arithmetic relations. If any relation is not satisfied by $\mathbf{x}, \mathbf{w}$, it aborts.
2. For RAM components, it initializes the memory using data from $\mathbf{x}$ or $\mathbf{w}$. It checks the data output by `read` accesses, and modifies the table according to `write` accesses. If any access operation is inconsistent, it aborts.

Theorem 1. *Define the relation by the proof of fixed-point XGBoost Training. Protocol 1 securely realizes $\mathcal{F}_{\text{CP}}$ in the $(\mathcal{F}_{\text{CVOLE}}, \mathcal{F}_{\text{RAMZK}})$ -hybrid model.*

Completeness. In the case of a pair of honest $\mathcal{P}$ and $\mathcal{V}$, the completeness is obvious since $\mathcal{P}$ honestly trains the model from the committed dataset and commits to the extended witness. Assume that the prover trains the model and proves ZK XGBoost both using fixed-point computation, the proof passes with probability 1.

Soundness. Our protocol makes black-box access to the underlying $\mathcal{F}_{\text{RAMZK}}$ and $\mathcal{F}_{\text{CVOLE}}$. Hence, a corrupted $\mathcal{P}$'s view can be simulated by constructing a simulator $\mathcal{S}$ who emulates the functionalities $\mathcal{F}_{\text{RAMZK}}$ and $\mathcal{F}_{\text{CVOLE}}$. In $\mathcal{F}_{\text{CVOLE}}$, it receives $\mathcal{D} = (\mathbf{x}, \mathbf{y})$, $\mathcal{T} = \{T_k\}_{k \in [m]}$ with $T_k = (\mathbf{f}_k, \mathbf{t}_k, \mathbf{w}_k)$, z , and their commitments from $\mathcal{P}$ and relay them to $\mathcal{F}_{\text{CP}}$. It also check whether the commitments successfully decommits to these values. If $\mathcal{F}_{\text{CVOLE}}$ aborts, it sends `abort` to $\mathcal{F}_{\text{CP}}$ and aborts itself. It receives IT-MACs from corrupted $\mathcal{P}$ and samples a correct VOLE correlation for committed data.

It emulates $\mathcal{F}_{\text{RAMZK}}$ with $\mathcal{P}$'s input and the XGBoost-FP relation, and $\mathcal{F}_{\text{RAMZK}}$ checks whether the XGBoost-FP relation is satisfied. If not, it sends **abort** to $\mathcal{F}_{\text{CP}}$ and aborts itself. Otherwise, it sends **PROVE – RECEIPT**.

Zero-Knowledge. $\mathcal{V}$ has no input to $\mathcal{F}_{\text{CP}}$ so the emulation of $\mathcal{V}$'s view is relatively simple. We construct a simulator $\mathcal{S}$, who emulates $\mathcal{F}_{\text{RAMZK}}$ and $\mathcal{F}_{\text{CVOLE}}$, and interact with corrupted $\mathcal{V}$ accordingly. It relays whatever it receives from $\mathcal{F}_{\text{CP}}$ to corrupted $\mathcal{V}$ and aborts when it aborts.

F Details of Certificate for Random Forest Training

In this section, we provide the detailed procedure for the certificate that verifies the correctness of random forest training. The procedure is summarized in Procedure 11. The algorithm takes as input a training data $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_n)$ with labels $\mathbf{y} = (y_1, \dots, y_n)$, a fitted forest $\mathcal{T} = \{T_k\}_{k \in [m]}$, where each tree T_k is represented by $(\mathbf{f}_k, \mathbf{t}_k, \mathbf{w}_k)$.

Procedure 11: CertForest

Parameters: Number of points n , features d , trees m , bins B , leaves $N = 2^h$ at depth h .
Input: Fixed-point training data $(\mathbf{x}, \mathbf{y})$, the fitted forest $\mathcal{T} = \{T_k\}_{k \in [m]}$, index sets $\{I_k\}_{k \in [m]}$, where $I_k \subset [n]$.
Output: 1 (accept) or 0 (reject).
1: $(\text{edges}, \text{binID}) \leftarrow \text{PreBinFP}(\mathbf{x}, B)$
2: **for** each tree $k \in [m]$ **do**
3: $\text{parse}(\mathbf{f}_k, \mathbf{t}_k, \mathbf{w}_k) \leftarrow T_k$
4: **for** each data point $i \in I_k$ **do**
5: compute the reached leaf $l_{k,i} \in [N]$ by evaluating T_k on $\mathbf{x}_i$
6: $\mathbf{l}_k \leftarrow (l_{k,i})_{i \in I_k}$
7: **assert** $\text{ValidateInference}((\mathbf{x}_i)_{i \in I_k}, \mathbf{f}_k, \mathbf{t}_k, \mathbf{l}_k) = 1$
8: $H_k \leftarrow \text{InitHistsLabel}((y_i)_{i \in I_k}, \text{binID}, \mathbf{l}_k)$
9: **assert** $\text{ValidateLeafWeightsLabel}(H_k, \mathbf{w}_k) = 1$
10: **assert** $\text{ValidateSplitsGini}(H_k, \mathbf{f}_k, \mathbf{t}_k, \text{edges}) = 1$
11: **return** 1 (accept)

Procedure 12: InitHistsLabel

Input: Labels $\mathbf{y} = (y_i)_{i=1}^n$, binID, leaf indices $\mathbf{l} = (l_i)_{i=1}^n$
Output: Histograms H
1: **for** $f \in [d]$, $\ell \in [2N - 1]$, $b \in [B]$, initialize $H[f][\ell][b] \leftarrow 0$.
2: **for** $i = 1$ **to** n **do**
3: **for** $f = 1$ **to** d **do**
4: $b \leftarrow \text{binID}[i][f]$; $\ell \leftarrow l_i + (N - 1)$
5: $H[f][\ell][b][y_i] \leftarrow H[f][\ell][b][y_i] + 1$
6: **for** $\ell = N - 1$ **down to** 1 **do** // propagate up to root
7: **for** $f = 1$ **to** d **do**
8: **for** $b = 1$ **to** B **do**
9: $H[f][\ell][b][0] \leftarrow H[f][2\ell][b][0] + H[f][2\ell + 1][b][0]$
10: $H[f][\ell][b][1] \leftarrow H[f][2\ell][b][1] + H[f][2\ell + 1][b][1]$
11: **return** H

Procedure 13: ValidateLeafWeightsLabel

Input: Leaf histograms H , weights $\mathbf{w} = (w_l)_{l=1}^N$
Output: Accept/Reject
1: **for** $l = 1$ **to** N **do**
2: $\ell \leftarrow l + (N - 1)$
3: $H_0^{\text{leaf}} \leftarrow \sum_{b=1}^B H[1][\ell][b][0]$ // use any feature f , as the sum is feature-independent
4: $H_1^{\text{leaf}} \leftarrow \sum_{b=1}^B H[1][\ell][b][1]$ // use any feature f , as the sum is feature-independent

```

5:    $w'_l \leftarrow \frac{H_1^{\text{leaf}}}{H_0^{\text{leaf}} + H_1^{\text{leaf}}}$ 
6:   assert  $w_l = w'_l$ 

```

Procedure 14: ValidateSplitsGini

Input: Node histograms H , resulting splits $(\mathbf{f}, \mathbf{t}) = (f_\ell, t_\ell)_{\ell=1}^{N-1}$, and **edges**

Output: Accept/Reject

```

1: for  $\ell = 1$  to  $N - 1$  do
2:   for  $f = 1$  to  $d$  do
3:      $H_0 \leftarrow \sum_{b=1}^B H[f][\ell][b][0]$ ;  $H_1 \leftarrow \sum_{b=1}^B H[f][\ell][b][1]$ 
4:      $H_{0,L} \leftarrow 0$ ,  $H_{1,L} \leftarrow 0$ 
5:     for  $b = 1$  to  $B$  do
6:        $H_{0,L} \leftarrow H_{0,L} + H[f][\ell][b][0]$ ;  $H_{1,L} \leftarrow H_{1,L} + H[f][\ell][b][1]$ 
7:        $H_{0,R} \leftarrow H_0 - H_{0,L}$ ;  $H_{1,R} \leftarrow H_1 - H_{1,L}$ 
8:        $H_L \leftarrow H_{0,L} + H_{1,L}$ ;  $H_R \leftarrow H_{0,R} + H_{1,R}$ 
9:        $\text{gain}[f][b] \leftarrow \left(1 - \left(\frac{H_0}{H_0 + H_1}\right)^2 - \left(\frac{H_1}{H_0 + H_1}\right)^2\right) - \frac{H_L}{H_L + H_R} \cdot \left(1 - \left(\frac{H_{0,L}}{H_L}\right)^2 - \left(\frac{H_{1,L}}{H_L}\right)^2\right) -$ 
 $\frac{H_R}{H_L + H_R} \cdot \left(1 - \left(\frac{H_{0,R}}{H_R}\right)^2 - \left(\frac{H_{1,R}}{H_R}\right)^2\right)$ 
10:    Retrieve  $b_\ell$  s.t.  $t_\ell = \text{edges}[f_\ell][b_\ell]$ 
11:    (Argmax check) assert  $\text{gain}[f_\ell][b_\ell] \geq \text{gain}[f][b]$  for all  $f \in [d], b \in [B]$ .

```

G Additional Experiments

G.1 Additional Fixed-Point XGBoost Experiments

Table 5: Breast Cancer ($n = 569$, $d = 30$): Accuracy and runtime of our fixed-point algorithm vs XGBoost.

Depth	Trees	FixedAcc	XGBAcc	$ \Delta $	T_{fixed} (s)	T_{xgb} (s)
4	50	0.9737	0.9766	0.0029	0.48	0.02
4	100	0.9708	0.9766	0.0058	0.66	0.02
5	50	0.9708	0.9766	0.0058	0.56	0.02
5	100	0.9708	0.9795	0.0088	0.74	0.02

Table 6: Covertypes - 50k ($n = 50000$, $d = 54$): Accuracy and runtime of our fixed-point algorithm vs XGBoost.

Depth	Trees	FixedAcc	XGBAcc	$ \Delta $	T_{fixed} (s)	T_{xgb} (s)
4	50	0.8186	0.8217	0.0021	7.26	0.12
4	100	0.8213	0.8235	0.0022	14.56	0.35
5	50	0.8203	0.8220	0.0017	9.32	0.35
5	100	0.8635	0.8610	0.0025	18.47	0.52

G.2 Discussion for Fixed-Point XGBoost Experimental Results

Accuracy parity. Across all five benchmarks (Credit Default and Covertypes-100k in Table 1, Breast Cancer in Table 5, Covertypes-50k in Table 6, Adult in Table 7) and the tested hyperparameter grid,

Table 7: Adult ($n = 45222$, $d = 104$): Accuracy and runtime of our fixed-point algorithm vs XGBoost.

Depth	Trees	FixedAcc	XGBAcc	$ \Delta $	T_{fixed} (s)	T_{xgb} (s)
4	50	0.8633	0.8711	0.0078	11.48	0.31
4	100	0.8645	0.8719	0.0074	22.75	0.45
5	50	0.8617	0.8708	0.0091	15.09	0.32
5	100	0.8615	0.8711	0.0096	30.20	0.49

Table 8: Running time for the proof of random forest training (Unit: minutes). First row shows dataset size n . Other parameters $h = 5, m = 64, d = 16$. Sparrow’s bagging time (30%) is taken out. Our proof contains tree structure hiding and more consistency checks for bit-decomposition and division.

n	16,384	65,538	262,144
Sparrow	17.64	45.01	155.19
Ours, LAN	27.6	66.3	218.1
Ours, WAN	29.2	69.4	229.3

our fixed-point training *tracks* floating-point XGBoost to within statistical error. Every configuration satisfies $|\Delta| \leq 1\%$. Notably, even on deeper/larger ensembles that are not presented here, the maximum observed gap remains under 0.01. These results confirm that adopting fixed-point arithmetic, together with our logit initializer, piecewise sigmoid, and fixed-point gain computation, does not degrade predictive accuracy in practice.

Performance. As expected, our fixed-point implementation is slower than XGBoost, which is a highly optimized C++ library leveraging vectorization, cache tuning, and parallelism. However, once we level the playing field a bit by pinning both systems to a single thread, the slowdown becomes very manageable. On the Credit Default dataset, slowdowns range between $20\times$ and $30\times$, while on the larger Covertype-100k dataset they range from $29\times$ to $42\times$. With parallelism neutralized, the residual slowdown is consistent with constant-factor overheads of a Python/NumPy prototype, which are plausibly reducible in a lower-level implementation. A rewrite in a low level language with all the relevant optimizations should substantially reduce constant factors and, with the already-demonstrated parity in accuracy, plausibly bring slowdowns into the low double digits or better. In any case, although the inefficiency is important to note, the absolute running time which is around 30 seconds even on the larger datasets is still very practical.

Small scale (Breast Cancer) as a counterpoint. On the very small Breast Cancer dataset (Table 5), the slowdown is more pronounced in relative terms, averaging around $32\times$. This reflects the fact that on tiny workloads, the fixed costs of our Python-level and fixed-point implementation dominate, while optimized libraries like XGBoost maintain nearly constant runtime overhead.

G.3 Comparing with Sparrow

Table 8 shows the running time of our proof of random forest training compared to Sparrow [PP24]. Sparrow relies on a memory-efficient zk-SNARK which supports public verifiability. Ours is interactive but our proof comes with stronger security guarantees: it hides tree structure for better privacy and includes more rigorous consistency checks for the proof of MSB and division to ensure soundness against malicious provers. Also, Sparrow is benchmarked in a much more powerful instance. Overall Sparrow’s prover time is slightly faster than ours.